ECE 51032: Computational Methods for Power System Analysis

Data-Center Connection Planning Game

Assignment and Student Tutorial

Purdue University

Fall 2026

This document has two parts. **Part I** is the official assignment: the rules, the scoring, what you submit, and how it is graded. **Part II** is the tutorial: it explains the game, the model, the tools, and strategy, and walks you through your first plan.

Part I: The Assignment

Assignment at a glance

Your role. Your team is a data-center operator in Texas. Each week you decide where to connect computing load to a model of the Texas grid, how many MW to request at each site, and whether to build a private generator or battery next to it. The instructor, acting as the grid operator, dispatches every team's plan together on the same network and scores each team in dollars. Your goal is the highest total score over the graded weeks.

What you submit. One CSV file per team per submission: two submissions a week for three weeks, six files in all. The CSV lists your sites and capacities (Rules of the game). At the end, each team also gives a short presentation and a report on what it did and why (Deliverables, grading, and conduct).

Key dates.

Week	Dates	Submission 1 due	Submission 2 due
1 (practice)	Nov 23–29	Thu Nov 26, 11:59 PM	Sun Nov 29, 11:59 PM
2 (graded)	Nov 30–Dec 6	Thu Dec 3, 11:59 PM	Sun Dec 6, 11:59 PM
3 (graded)	Dec 7–13	Thu Dec 10, 11:59 PM	Sun Dec 13, 11:59 PM

Submission 1 results, and the Submission 2 data, are released before each Submission 2 is due.

Teams. Teams have one to three students, and you form your team yourselves. The same standard applies whether a team has one student or three. Each team submits one CSV per submission, uploaded by one member, with the same `team_id` all game long.

Getting help.

- Office hours: **[Instructor: times and places]**.
- Questions and discussion: the course Discord/Slack channel (the link will be provided).
- Code, data, and this document: the course GitHub repository (section 4).

Rules of the game

Schedule. The game runs for three weeks. Every week plays the same game on the same data: a plan for the next ten years, made in two submissions (fig. 1).

- Week 1 is practice, without energy cost: it is scored so that you can see how the evaluation works, but it does not count.
- Week 2 is graded and, like Week 1, has no energy cost.
- Week 3 is graded and charges energy at the locational marginal price (LMP) in both submissions.

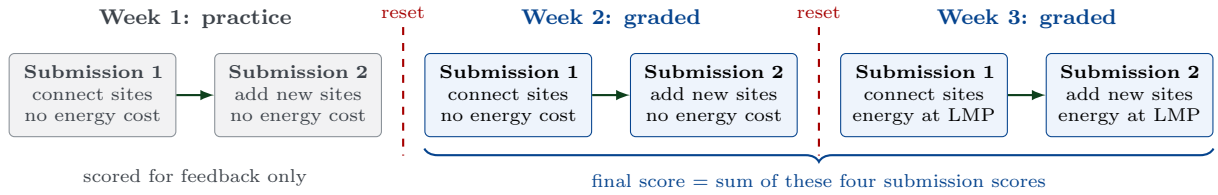


Figure 1: The submission timeline. Arrows: the load connected in Submission 1 becomes part of the base load for Submission 2 of the same week. Everything is removed between weeks.

Within a week.

- Submission 1 connects sites to the grid with the representative base load.
- In Submission 2, the load that every team connected in Submission 1 is part of the base load: it is already on the grid and is not re-evaluated. You submit only new sites, and they compete for the room that is left.
- Each site is scored once, in the submission where it is built, for its ten-year value.
- Everything resets at the start of the next week, so you can improve your strategy based on what you saw the week before.

Time model. Every submission uses the same representative data for the next ten years: five typical days of 24 hourly periods, 120 periods in all. Each day stands for a number of days of the year (the peak-load day stands for one), so together the five days make up one typical year. We assume that each of the next ten years looks like this typical year: the same loads, the same network, and the same generators. A site, once connected, runs for all ten years. Batteries follow a daily cycle: they start each day half full and must end it half full. The grid operator decides how your generator and battery run in every period, and they can serve only your own load (sections 1 and 3).

Submission format. One CSV per team with exactly this header and one row per site:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
```

- One `team_id` on every row; every `site_id` unique; every `bus_id` must exist in the representative data package.
- 4 Capacities: Connection capacity $C \geq 0$, generator capacity $0 \leq G \leq 0.5\,C$, and storage either $P = B = 0$ or $B = 4P$. A generator may run at most 2 hours a day at full output ($2G$ MWh per representative day), a stand-in for the run-hour limits in air permits for on-site engines [26, 9].
- In Submission 2, list only new sites. Reusing a `site_id` of your Submission 1 makes the whole file invalid.
- File name: follow the format team{x}_week{x}_sub{x}.csv, e.g., team07_week2_sub1.csv. Upload to: BrightSpace.

Details and examples are in section 8.

Validity. Run `ercot-game-check-csv` with `--artifact-dir` on the file you upload. A file that fails validation scores 0 for that submission.

Scoring and evaluation

Score of one submission (in US dollars):

$$\text{score} = \text{IT value} - \text{energy cost} - \text{generator operating cost} - \text{generator capital} - \text{storage capital} - \text{site cost}.$$

Term	Value	How it is charged
IT value	\$1,000 per MWh	Paid for every MWh of your load that is actually served, over ten years.
Energy cost	Locational Marginal Price (LMP) at your site	Grid purchases times the LMP, over ten years. Week 3 only; Weeks 1 and 2 have no energy cost.
Generator operating cost	\$80 / \$150 / \$250 per MWh	Piecewise: first 25% of capacity at \$80, next 50% at \$150, last 25% at \$250, over ten years.
Generator capital	\$2.0M per MW	On generator capacity, once, when it is built.
Generator run hours	2 h per day	Each representative day, a generator of G MW may produce at most $2G$ MWh.
Storage capital	\$400k per MW + \$250k per MWh	Once, when it is built. A 4-hour battery costs \$1.4M per MW of power.
Site cost	\$25M per MW	On connection capacity, once, when the site is built.

Ten years, in present value. Each representative hour is weighted by the number of days its day stands for, so the 120 hours make up one year of 8,760 hours, and ten such years make 87,600 hours. All money in the score is a present value: a dollar earned in year y counts as $1/1.07^y$ dollars today (a 7% discount rate, the rate U.S. federal guidance long used for returns on private capital [27]), because money later is worth less than money now. Over ten years this multiplies every per-MWh amount by $\frac{1}{10}\sum_{y=1}^{10} 1.07^{-y} = 0.702$. So one MW served in every hour for ten years earns $87{,}600 \times \$1{,}000 = \$87.6\text{M}$, which is \$61.5M in present value. Energy and generator fuel are discounted the same way. Capital is paid once, up front, so it is not discounted. Section 7 gives the formula. The scoring-rules file `data/teaching/dispatch_scoring_rules_representative_v1.json` is the source of these numbers.

Final score. Your final score is the sum of your four submission scores in Weeks 2 and 3.

One joint evaluation. All valid plans are dispatched together in one DCOPF per operating period, so whether your load is served depends on where the other teams build. If the combined plan does not fit the grid:

- every team is served up to a common MW level, and a team that asked for less is served in full;
- teams that can still grow keep rising until they are blocked;
- a team that would be served less than half of its request gets nothing.

Capital is always charged on what you *requested*, served or not. The rule is explained in section 2.2 and specified in appendix C.

What you get back. After each submission: your score breakdown, every team's service fraction, whether the projection was triggered, and an interactive grid map with the total connection capacity per bus (section 14). Other teams never see your individual plan.

Preview versus official score. `ercot-game-preview` gives an approximate score for your plan alone. Only the official evaluator, run by the instructor with every team's plan, produces your score.

Deliverables, grading, and conduct

What you will learn. By the end of the game you should be able to

- formulate a power-system planning decision as an optimization problem and solve it with an optimization package (DCOPF, hosting capacity);
- read locational marginal prices and congestion as signals for where new load can connect;
- size distributed generation and storage against network limits and energy prices;
- reason about a planning decision whose outcome depends on what other players do, as in an electricity market;
- use computational tools, including AI assistants, critically and reproducibly.

Deliverables. The six CSV files, one per submission, uploaded by the deadlines in Assignment at a glance, and at the end a short presentation (tentatively scheduled for the last week) and a report explaining what your team did and why.

Grading. The assignment counts for 40 points of the final grade:

- 30 points for the presentation, the report, and your work;
- 10 points for your final game score S, the sum of your four graded submissions:
  - 5 absolute points: $5 \times \min\big(1, \max(0, S/(2B))\big)$, where B is the score of a simple reference plan: one 300 MW site at bus 6147, with no generator or battery, built in Submission 1 of Weeks 2 and 3 and evaluated alone, with nothing added in Submission 2. It scores \$10.96B in Week 2 and \$10.63B in Week 3, so $B =$ \$21.59B and full absolute points need $S \geq$ \$43.2B. These points do not depend on other teams, so everyone can earn all of them, and nobody gains by hurting another team.
  - 5 rank points, by your rank among all teams:

Rank	Top 20%	20–40%	40–60%	60–80%	Bottom 20%
Points	5	4	3	2	1

Bonus, up to 5 points:

- Top three final scores: 3, 2, and 1 points.
- Prediction or insight, up to 2 points: the most accurate prediction of your own service fraction, stated with your Week 3 Submission 1, or the best explanation in your presentation of why something happened in the game.

Week 1 is practice and earns no points.

Late and invalid submissions. Late submission will not be accepted, sinceall teams are evaluated together, and a late file cannot be added to a round that has already been run. A late submission is treated as a zero.

Working with other teams. Teams may talk to or coordinate with each other. In any case, do not share your CSV, your exact plan, or shared chat links that contain it with another team.

Use of AI. You may use AI tools for any part of this assignment (section 5). Your team is responsible for every number you submit and must be able to explain its plan. Do not paste passwords, tokens, or other teams' information into an AI tool.

Academic integrity. Purdue's academic integrity policy applies. Each team's plans must be its own work.

Problems with your results. If you think your submission was evaluated incorrectly, contact Minghao via mmou@purdue.edu within 7 days of the results release.

Part II: The Tutorial

Quick start

1. Read Part I, the assignment (Assignment at a glance to Deliverables, grading, and conduct).
2. Set up the GitHub repository (section 4).
3. Try the AI-agent walkthrough on the IEEE 14-bus demo (section 6).
4. Read sections 1 and 2 so you know what earns points.
5. Explore the representative data (section 7).
6. Write your plan as a CSV and run the checker on it (section 8).
7. Preview the plan, change one thing at a time, and repeat (sections 9 and 11).
8. Go through the checklist in section 13 and submit.
9. When the results come out, read them as described in section 14.

Contents

Part I: The Assignment 1
Assignment at a glance 1
Rules of the game 2
Scoring and evaluation 4
Deliverables, grading, and conduct 6

Part II: The Tutorial 8
1 The game in one page 10
1.1 What you decide . . . 10
1.2 Schedule and data . . . 10
2 How your score is computed 11
2.1 A worked example . . . 12
2.2 You are not the only player . . . 12
3 How the grid operator runs your plan 13
3.1 Storage . . . 14
3.2 Prices . . . 14
4 Set up your computer 14
5 Using AI for this assignment 15
5.1 Which kind of tool . . . 15
5.2 How to write a good prompt . . . 16
5.3 Work in a loop . . . 16
5.4 Check what the AI tells you . . . 16
5.5 Privacy . . . 17
6 Walk through the workflow with an AI agent 17
6.1 Step 1: understand the assignment . . . 18
6.2 Step 2: look at the data . . . 18
6.3 Step 3: hosting capacity . . . 19
6.4 Step 4: write and check a plan . . . 19
6.5 Step 5: preview the plan . . . 20

6.6 Step 6: what if a rival copies you? . . . 20
7 Explore the data **21**
7.1 Hosting capacity: how much room a bus has . . . 22
8 Write and check your CSV **23**
9 Preview your plan **24**
10 Putting it together: your first plan **25**
11 Develop a strategy **25**
11.1 A two-team example . . . 26
11.2 Run controlled experiments . . . 27
12 Troubleshooting **28**
13 Before you submit **28**
14 After each submission: reading your results **29**
15 Frequently asked questions **29**
16 Glossary **31**

Appendices 32

A Hosting capacity in detail **32**
A.1 Formulation . . . 32
A.2 Computing it . . . 34
B The evaluator in full **35**
B.1 Solving a day quickly: Benders decomposition . . . 36
C The fair projection in detail **38**
C.1 How a team's service is split among its sites . . . 38
C.2 The four steps . . . 38
C.3 Steps 1 and 2 in detail . . . 39
C.4 Why the rule is max-min fair and strategy-proof . . . 40
D Where the numbers come from **41**
E Working as a team **42**

This part walks you through the assignment from start to finish: what the game is, how your plan is scored, how to explore the data, how to write and check your submission, and how to use AI tools along the way. The numbers come from the scoring-rules files in `data/teaching/`, and `docs/HYPERPARAMETERS.md` lists every game parameter with its source.

1 The game in one page

Your team is a data-center operator. You choose where to connect computing load to a public-data-informed planning model of the Texas grid, how large (in MW) connection is, and whether to build your own generation and battery storage collocated with it. The instructor, acting as the grid operator, then runs one DCOPF dispatch with every team's plan in it, and scores each team. The grid is the public ACTIVSg2000 test system [23]: 2,000 buses, 3,206 branches, and 544 generators, stored in the MATPOWER case format [31].

1.1 What you decide

Your plan is a list of *sites*. For each site you choose four numbers:

Decision	Unit	Meaning
Bus	–	Where the site connects to the network (a `bus_id`).
Connection capacity C	MW	Your requested computing load cap at this site.
Generator capacity G	MW	Optional private generator at the site, at most $0.5\,C$.
Storage power P / energy B	MW / MWh	Optional battery: either both zero, or $B = 4P$.

You only choose sites and four capacities. The grid operator decides how your generator and battery run in each operating period. Your generator and battery serve only your own load: they never export to the grid or to another team.

1.2 Schedule and data

The schedule and the timeline are set out in Rules of the game (fig. 1). In short: three weeks, each playing the same ten-year game in two submissions; Week 1 is practice, Weeks 2 and 3 count, and only Week 3 charges energy.

Every submission uses the same representative data for the next ten years, in `data/processed/activsg2000_representative_v1/`: five typical days of 24 hourly periods, 120 periods in all. They were picked from a year of hourly load data: four days that each stand for a group of similar days, plus the peak-load day, which stands for itself.

Day	Periods	Stands for (days per year)	Daily energy (GWh)	Peak (GW)
1 (winter)	0–23	83.77	1,569	69.4
2 (spring)	24–47	99.73	1,493	66.2
3 (summer)	48–71	82.77	1,653	72.9
4 (late summer)	72–95	97.73	1,729	76.1
5 (peak-load day)	96–119	1	1,780	78.2

The days sum to 365. Your CSV uses one row per connection site; the evaluator applies that plan to all 120 periods.

From 120 hours to ten years

Period t belongs to a day that stands for d_t days of the year, so it stands for d_t hours of each year ($\sum_t d_t = 8{,}760$) and $10\,d_t$ hours of the ten years (87,600 in all). Every per-MWh amount (IT value, energy, generator fuel) earned in period t is counted in present value:

$$\text{value} = \sum_t (\$ \text{ per hour in period } t) \times d_t \times A, \qquad A = \sum_{y=1}^{10} \frac{1}{1.07^y} = 7.02,$$

where A adds up the ten years, each discounted at 7% per year. Since $A = 0.702 \times 10$, a present value is 0.702 times the plain ten-year total. For one MW served in every hour: $87{,}600 \times \$1{,}000 \times 0.702 = \$61.5\text{M}$. (The rules file stores the weights $d_t A$ for each period.) Capital is paid once, up front, and is not discounted. With `daily\_exact\_cycle`, a battery starts each representative day at $B/2$ and must end it at exactly $B/2$.

Key idea

In Submission 2, the load that all teams connected in Submission 1 is part of the base load. A site added in Submission 2 earns the same ten-year value as one built in Submission 1, but it competes only for the room that is left.

Submission 2 can cost more than it earns

Submission 1 may leave little or no room on the grid. A Submission 2 site that would be served less than half of its request is served nothing, but it still pays its full site cost, so a Submission 2 score can be billions of dollars below zero. Submitting no new sites scores exactly zero. Before you submit, recompute hosting capacity on the Submission 2 data your instructor releases (section 7.1) and preview your plan on it (section 9).

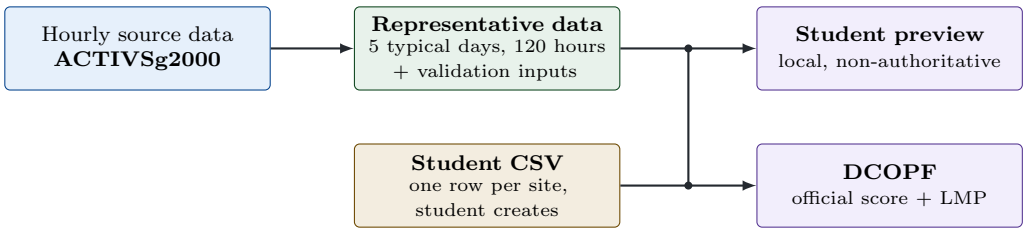


Figure 2: What you receive and what you submit. The released package gives you the network plus scenario context and validation inputs. You create the plan (CSV submission), and can preview it yourself, but only the official evaluator produces your score.

2 How your score is computed

For each submission, your score in US dollars is

$$\begin{aligned}\text{score} = \text{IT value} &- \text{energy cost} - \text{generator operating cost} \\ &- \text{generator capital} - \text{storage capital} - \text{site cost}.\end{aligned}$$

The coefficients are in the table in Scoring and evaluation: IT value \$1,000/MWh served, energy at the LMP in Week 3 only, a generator curve of \$80/\$150/\$250 per MWh, all over ten years, and capital of \$25M/MW for a site, \$2.0M/MW for a generator (which may produce at most two hours' worth of its capacity per day), and \$400k/MW plus \$250k/MWh for storage, charged once when built. Storage has no operating cost. This section shows what those numbers mean for a plan.

2.1 A worked example

Suppose you build one 50 MW site and nothing else, and all of its load is served in every period. Baseline LMPs are about $18/MWh.

In Week 1 or Week 2 (no energy cost):

$$\begin{aligned} \text{IT value} &= 50 \times 87{,}600 \times 1{,}000 \times 0.702 = \$3{,}076.3\text{M} \\ \text{site cost} &= 50 \times 25\text{M} = \$1{,}250.0\text{M} \\ \text{score} &= \$1{,}826.3\text{M} \end{aligned}$$

In Week 3 (energy at the LMP):

$$\begin{aligned} \text{energy cost} &\approx 50 \times 87{,}600 \times 18 \times 0.702 = \$55.4\text{M} \\ \text{score} &\approx \$1{,}770.9\text{M} \end{aligned}$$

Each served MW earns about $61.5M (present value) over ten years against a one-time site cost of $25M, so a site breaks even once about 41% of its load is served.

Key idea

Energy is cheap compared with the value of serving load ($18 against $1,000 per MWh). What decides your score is whether the grid can actually deliver power to your site. Load the network cannot serve still pays the full site cost.

Try it

A team adds a 30 MW site in Submission 2 of Week 3, and it is fully served at an LMP of about $18/MWh. Estimate its score. *(Answer:* $30 \times 87{,}600 \times (1{,}000 - 18) \times 0.702 - 750M \approx \$1{,}063M.$*)*

2.2 You are not the only player

The worked example in Section 2.1 quietly assumed that all of your load is served. Whether it is depends on more than your own plan. Every team reads the same rules and does the same arithmetic, so every team is tempted by the same plan: build big, build early, and pick buses (sites) where the network looks strong. Each of those plans can be fine on its own, and your preview, which runs without the other teams, will say so. But the spare capacity of the grid is shared. If too many teams crowd the same part of the network, the combined plan has no feasible dispatch, even though no single plan caused the problem.

> *"The hard thing in life is getting you to come out of your own selves a bit, realizing it's 'not all about you.'"*

When the combined plan of all teams is infeasible, the evaluator has to decide who gets served. It applies a fair projection based on the *uniform rule*:

1. Service fractions. Each team i gets a single fraction $\alpha_i \in [0, 1]$ that scales the connection capacities of all its sites in every period.
2. Equal level. A common level t (in MW) rises from zero, and team i is served $\min(r_i, t)$, where r_i is its total requested connection capacity. The level stops at the largest t for which the joint dispatch is feasible in every operating period. A team that asked for less than t is served in full.
3. Fill the slack. If some teams can still grow without hurting a team that is already stuck, their level keeps rising until every team is either fully served or blocked.

4. Minimum allocation. A team that would be served less than 50% of its request gets nothing. The largest such request is dropped first, and the allocation is recomputed for the rest.

This changes the question you should ask. Your served load, and so whether you clear the break-even points above, depends on what everyone else builds. A bigger request does not buy you priority: you are never served more than the common level, and a request so large that it would be served less than half is dropped entirely, while capital is still charged on the full capacity you submitted. So the question is not only "how much load can my site take?" but also "how much room will be left for me once the other teams have built, and what do they expect me to do?"

Section 11.1 works through a stylized two-team example at two neighboring buses.

Think about it

Suppose you expect most teams to crowd the best-connected buses. What do you gain and what do you risk by choosing a less popular location instead? What would the Week 1 results tell you about how other teams actually behave?

3 How the grid operator runs your plan

The evaluator solves a DCOPF on the ACTIVSg2000 network for each of the 120 representative hours, with all teams' sites added. The existing network, generators, and base loads come from the representative data package (in Submission 2, the base load also contains the Submission 1 sites). Figure 3 shows what one row of your CSV adds at a bus k.

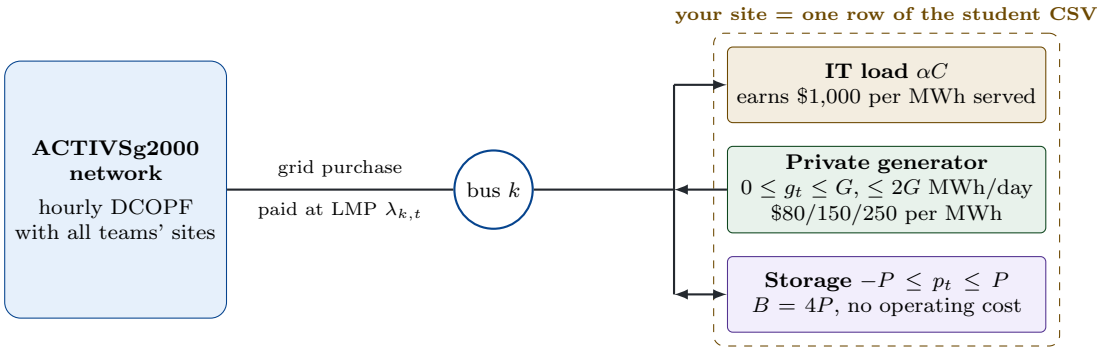


Figure 3: One site in the dispatch. The site draws αC of IT load; your generator and battery can cover part of it, and the rest is bought from the grid at the bus LMP.

For a site at bus k in period t, the evaluator adds:

- a fixed load αC (normally $\alpha \in [0, 1]$, cf. Section 2.2);
- a private generator $0 \le g_t \le G$ with the piecewise cost in Scoring and evaluation;
- a battery with signed power p_t (positive = discharge), limited by $-P \le p_t \le P$.

Your grid purchase rate in period t is $\alpha C - g_t - p_t$. The generator and battery can only cover your own team's load at that bus; they cannot push power back into the grid.

3.1 Storage

Each battery follows the model below, with hourly steps and 100% efficiency. The signed variable $p_{d,h}$ is a power rate in MW: positive means discharge and negative means charge. Let $\Delta t = 1$ hour.

$$e_{d,h} = e_{d,h-1} - \Delta t\, p_{d,h}, \qquad 0 \le e_{d,h} \le B, \qquad -P \le p_{d,h} \le P, \qquad \Delta t\, p_{d,h} \le e_{d,h-1},$$
$$e_{d,0} = 0.5B, \qquad e_{d,24} = 0.5B \qquad \text{for each representative day } d.$$

The battery starts every representative day half full and must end it at exactly half full after 24 hours (fig. 4). There is no state-of-charge carryover from one day to the next. Discharge at a bus cannot exceed your own load there.

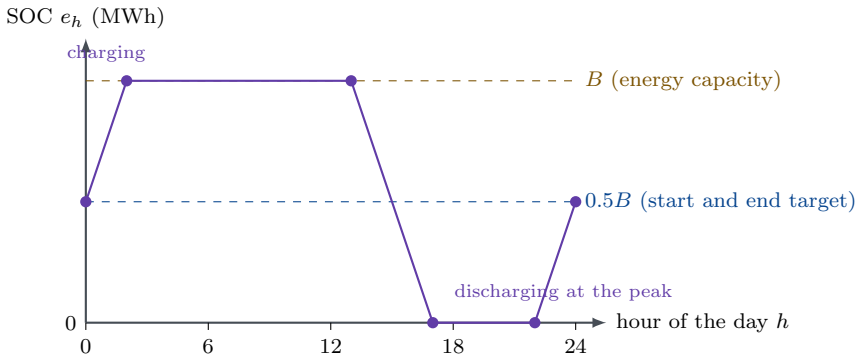


Figure 4: An illustrative one-day battery schedule with $B = 4P$: it starts at $0.5B$, stays between 0 and B, moves by at most P MWh per hour, and returns to exactly $0.5B$ at the end of the day. The evaluator chooses this schedule independently for each representative day.

3.2 Prices

The LMP at your bus is the dual of that bus's power balance in the joint dispatch, clipped at zero. In Week 3, energy is settled against LMPs computed from the actual joint dispatch containing all accepted teams. Optional baseline LMP artifacts may be generated for diagnostics. With teams' load added, lines can congest and the LMP at your bus can rise.

Try it

A battery has $P = 5$ MW and $B = 20$ MWh. It starts a representative day at the required state of charge. Can it discharge at 5 MW for two hours in a row? What is the state of charge after each hour? *(Answer: it starts at 10* MWh*, is at 5* MWh *after one hour and 0* MWh *after two; it cannot discharge in a third hour until it has charged. It must also return to exactly 10* MWh *by the end of the day.)*

4 Set up your computer

You need Python 3.11 or newer and the course repository. From a terminal:

```
git clone <course-repository-URL> ercot-game
cd ercot-game
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install -e ".[tutorial]" pandas
```

Run every command in this tutorial from the repository root with the virtual environment active. Check the installation by validating the example submission (section 8 explains the output):

```
ercot-game-check-csv --file examples/eight_team_round/submissions/team-01.csv
```

The files you will use most:

Path	What it is
`docs/HYPERPARAMETERS.md`	Every game parameter, with its value and source.
`data/processed/activsg2000_representative_v1/`	Representative data: five typical days, 120 hourly periods.
`data/teaching/dispatch_scoring_rules_representative_v1.json`	Scoring coefficients and the ten-year weights used by the preview and the evaluator.
`ercot-game-check-csv`	Checker for your CSV (also `scripts/check_submission.py`).
`ercot-game-preview`	Approximate scoring run for your plan.
`docs/student_tutorial/notebooks/`	Small companion notebooks (see below).

The notebooks in `docs/student_tutorial/notebooks/` use tiny made-up fixtures, not the real data. They are useful for seeing the mechanics in isolation: `03_three_bus_dcopf.ipynb` solves a three-bus DCOPF with the same solver the evaluator uses, `04_storage_soc.ipynb` traces a battery's state of charge, and `05_csv_validation.ipynb` calls the checker from Python.

5 Using AI for this assignment

You are encouraged to use AI tools such as ChatGPT, Claude, Gemini, GitHub Copilot, Claude Code, or Codex. You are still responsible for every number you submit and must be able to explain your plan. This section is for anyone who has not used these tools much. If you are already familiar with AI tools, you can skip this section.

5.1 Which kind of tool

- **Chat assistants** (ChatGPT, Claude, Gemini in a browser) are best for explaining concepts, writing short scripts, and decoding error messages. They only see what you paste in. Most of them can be used for free. However, without subscriptions, we may be limited to only the use of basic models.
- **Coding assistants/AI agents** [recommended] (Claude Code, Codex, Copilot in your editor) can read the GitHub repository and run commands for you. They are faster once you are comfortable, but check what they change. Most of them require subscriptions. All the demos in the tutorials used coding assistants, since it is not easy for char assistants to manipulate local files.[1]

If you are new to this, start with a chat assistant and copy code into your own files. Each tool has an official getting-started guide:

[1] There are in fact some advanced tools that allow chat assistants to get access to and manipulate local files. You can search for those tools online. However, using those tools comes with risks of getting your account suspended.

Tool	Kind	Getting started
ChatGPT	Chat	What is ChatGPT: FAQ
Claude	Chat	Get started with Claude
Gemini	Chat	Use Gemini Apps
GitHub Copilot	Coding (editor)	Quickstart for GitHub Copilot in your IDE
Claude Code	Coding (terminal)	Claude Code quickstart
Codex	Coding (terminal)	Codex CLI

GitHub Copilot is free for verified students, see Access GitHub Copilot for free as a student.

5.2 How to write a good prompt

The AI knows DCOPF but knows nothing about this assignment. Give it four things:

- the **goal**,
- the **context** (paste file headers, a few rows, error messages),
- the **constraints**,
- the **form** of answer you want.

Weak prompt

Write code to analyze the data.

Better prompt

I am working in Python with pandas. I have a file `branch_limits.csv` with columns `branch_id, from_bus_id, to_bus_id, reactance_pu, thermal_limit_mva, status`. Here are its first three rows: [paste]. Write code that, for each bus, adds up `thermal_limit_mva` over in-service branches (`status == 1`) connected to that bus, and prints the 20 buses with the largest total. Explain each line briefly.

5.3 Work in a loop

Treat the AI as a pair programmer. Ask for something small, run it, and paste back exactly what happened, including the full error message. Repeat until it works, then ask it to explain the final version so you understand it.

Ask AI

I ran your script and got this error: `KeyError: 'thermal_limit'`. The column names are exactly: `branch_id,from_bus_id,to_bus_id,reactance_pu,thermal_limit_mva,status`. Please fix the script.

Example AI reply

The column is called `thermal_limit_mva`, not `thermal_limit`. Change line 6 to `branches["thermal_limit_mva"]` and run it again.

5.4 Check what the AI tells you

AI tools are often confidently wrong about assignment-specific details (and this is the reason why you need to provide enough context to AI agents). They might invent a CSV column, assume you schedule the battery yourself, or use made-up prices. Before you rely on an answer:

- **Run it.** Code that has not run is a guess.
- **Check the numbers** against this tutorial and the rules JSON.
- **Try a case you can do by hand**, such as the worked example in section 2.
- **Ask for its assumptions**: "What did you assume that is not in the files I gave you?"

5.5 Privacy

Do not upload passwords, access tokens, or other credentials. Your plan is your team's competitive information, so do not share it with other teams,[2] including through shared chat links. Public data from the repository is fine to paste.

6 Walk through the workflow with an AI agent

This section shows the whole workflow, from reading the rules to a joint evaluation with a rival team, done by talking to an AI agent. It uses a small IEEE 14-bus demo that ships with the repository (`data/teaching/ieee14_demo_representative`): 14 buses, 20 lines, 5 generators, and the same five representative days (120 hourly periods) and ten-year scoring as the game. Every command runs in a few seconds, so you can repeat the session yourself. The demo is for practice only: the game itself runs on `ACTIVSg2000`, and the demo's numbers do not carry over.

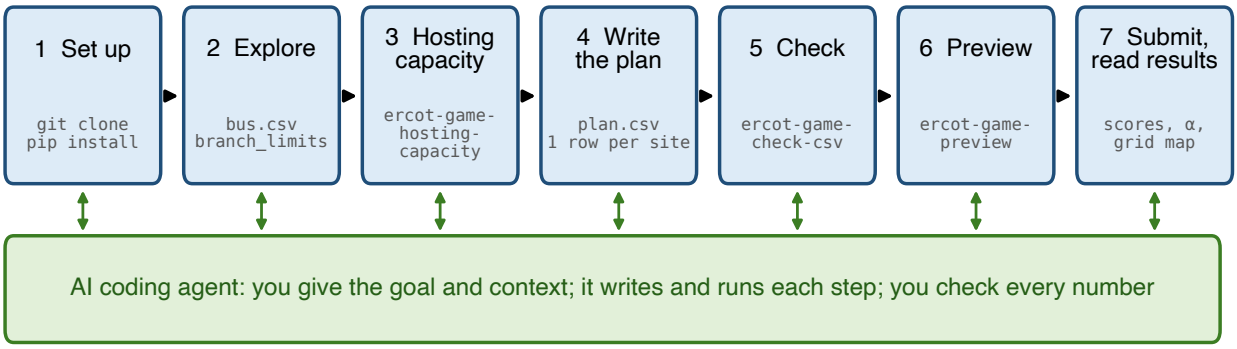


Figure 5: The workflow in the repository, with an AI agent in the loop. Each step has its own section: set up (section 4), explore (section 7), hosting capacity (section 7.1), write and check the plan (section 8), preview (section 9), and read the results (section 14). The six prompts below walk through steps 2–7.

How the session was run. A coding agent (Claude Opus 5.5) was started in a fresh clone of the repository with the package installed (section 4). It had not seen the repository before. The six prompts below were typed one after another in the same session. The replies are real but shortened, with `[...]` marks a cut. The complete prompts and replies are in `docs/student_tutorial/ai_walkthrough/`. To start an agent yourself, open a terminal in the repository folder and run, for example, `claude` (Claude Code) or `codex` (Codex CLI).

Watch how each prompt is written. It states the goal, names the files and commands, sets constraints ("do not change any files", "under 150 words"), and asks for the form of the answer ("a table", "show the commands"). After every reply, a box lists what you should check yourself before you rely on it.

[2] However, it is possible to cooperate with other teams. It is also an interesting direction to explore.

6.1 Step 1: understand the assignment

Prompt 1

I just cloned this repository for my ECE 51032 course project. Read the README and Part I of `docs/student_tutorial/main.tex` (context), then explain in under 150 words (constraint): what my team submits, how it is scored, and which commands I will use. (deliverables) Do not change any files (constraint).

Agent reply (shortened)

What you submit: one CSV per submission, two a week for three weeks (six files in all). [...] The rules are $G \leq 0.5C$ and storage either zero or $B = 4P$. Submission 2 lists only new sites. You also give a final presentation and a report.
Scoring: score = IT value ($1,000/MWh served, over 10 years) − energy cost (charged at the LMP, Week 3 only) − generator operating cost − capital costs [...]. Your final score is the sum of the four Week 2 and 3 submissions. [...]
Commands: `ercot-game-check-csv` [...] and `ercot-game-preview`. Note: the README describes an older six-hour setup ($6,500/MWh). Follow Part I.

Check it yourself

When this session was recorded, the README still described an earlier version of the game; it has since been updated. The agent noticed that two documents disagreed and said which one it trusted, which is exactly what you want. But it left out a tool you will need, `ercot-game-hosting-capacity`. Compare its summary with Rules of the game and Scoring and evaluation rather than trusting a summary.

6.2 Step 2: look at the data

Prompt 2

We will practice on the small IEEE 14-bus demo package in `data/teaching/ieee14_demo_representative` before touching the real grid. Summarize it: number of buses, lines, and generators, total load, the number of operating periods, and the five lines with the smallest thermal limits. Show the commands you ran.

Agent reply (shortened)

Buses: 14. **Lines:** 20, all in service. **Generators:** 5, with 772.4 MW total capacity [...]. **Load:** the base load is 259.0 MW. Across the periods it ranges from 224.5 to 301.7 MW [...]. **Operating periods:** 120. That is 5 representative days of 24 one-hour periods. The day weights add up to 365, and the peak day has a weight of 1. **Five smallest thermal limits:** 12–13 (20 MVA), 10–11 (25), 13–14 (25), 6–11 (30), 6–12 (30). br16 (9–10) is also at 30 MVA, so the 4th and 5th places are a three-way tie at that limit.

Check it yourself

Open `branch_limits.csv`: four lines are at 30 MVA, not three. The agent missed 9–14, which matters later because it is one of the lines that limits buses 13 and 14. Small slips like this are common; spot-check any table an agent gives you.

6.3 Step 3: hosting capacity

Prompt 3

Use `ercot-game-hosting-capacity` to compute the hosting capacity of every bus in the demo package over all 120 hours. Give me a table of each bus's minimum hosting capacity, sorted from largest to smallest, and say in one sentence what this number means. Which two buses would you shortlist for a data center, and why?

Agent reply (shortened)

```
ercot-game-hosting-capacity \
  --artifact-dir data/teaching/ieee14_demo_representative \
  --horizon 120 --period-count 120 --output-dir <tmp>/hc14
# backend highs, bus_count 14, completed_point_count 1680, failed_point_count 0, wall_time
    ↪ 1.6 s
```

The columns are in the order of `bus_ids` in the metadata file (`1,10,11,...`), not numeric order, so I matched them up by that list.

Bus	1	2	5	4	3	7	8	6	9	13	11	10	12	14
Min HC (MW)	471	429	311	241	180	178	178	169	103	51	45	45	43	38

What it means: a bus's minimum hosting capacity is the most new load you could add at that bus alone [...] that the grid can still serve in every one of the 120 hours [...]. For every bus the lowest value falls in hour 109. **Shortlist: buses 1 and 2.** [...] **One caution:** these numbers are for one bus at a time, and they don't add up. Every team will see the same table, so buses 1 and 2 may get crowded.

Check it yourself

Re-run one bus yourself (add `--bus-id14`) and compare. The agent caught a real trap, the column order of the output file. Its caveat, that every team sees the same table, is the most important line of the reply.

6.4 Step 4: write and check a plan

Prompt 4

Write a plan for team-demo with one site at each of the two buses you shortlisted, each at about half of its hosting capacity, with no generator and no battery. Save it as `runs/demo/plan.csv` and check it with `ercot-game-check-csv` against the demo package. Then make a copy with a mistake, a generator larger than its connection, and show me what the checker says.

Agent reply (shortened)

I sized each site at about half of its bus's minimum hosting capacity: 235 MW at bus 1 and 214 MW at bus 2, with no generator or battery.

```
VALID: runs/demo/plan.csv
team=team-demo sites=2 rows=2 buses=2 generators=0 storage=0 horizon=None

INVALID: runs/demo/plan_bad.csv
- row 2: generator capacity cannot exceed 0.5 x connection capacity (300 MW > 117.5 MW)
```

The checker's limit is stricter than "no bigger than the connection": a generator can be at most half the connection capacity [...]. That is the $G \leq 0.5C$ rule from Part I.

Check it yourself

The agent saved the broken copy *next to* the real plan, in `runs/demo/`. The preview runs every CSV in a folder together, so keep each candidate in its own folder (the agent fixed this itself in the next step).

6.5 Step 5: preview the plan

Prompt 5

Preview the valid plan for Week 3, when energy is charged at the LMP (use `--week3`). Report the share of load served, the score, and the term that dominates it. Is the plan above break-even?

Agent reply (shortened)

The preview needs a folder of submissions, and `runs/demo/` also holds `plan_bad.csv`. So I copied the valid plan into its own folder. [...]

Term	$
IT value	+27.63 B
Energy cost (LMP)	−0.83 B
Site cost (449 MW × $25M)	−11.23 B
Score	**+15.57 B**

Share of load served: 100%. [...] **Break-even:** [...] That puts break-even at about 44% service, and the plan is far above it.

Check it yourself

Redo the arithmetic: $449 \times 87{,}600 \times \$1{,}000 \times 0.702 = \$27.6$B, as reported. But the break-even is wrong: with $1.85M of energy per MW it is $25/(61.5 - 1.85) \approx 42\%$, not 44%. Always redo an agent's own back-of-envelope numbers.

6.6 Step 6: what if a rival copies you?

Prompt 6

Other teams will read the same hosting-capacity table. Simulate a rival, team-rival, that copies our plan exactly (same buses, same sizes), and run the joint evaluation with `ercot-game-batch` on both plans for Week 3. What service fraction does each team get, and what does that mean for our plan? Finally, list every assumption you made in this session that I should check myself.

Agent reply (shortened)

```
ercot-game-batch run --submissions-dir runs/demo/joint \
  --artifact-dir data/teaching/ieee14_demo_representative \
  --rules data/teaching/dispatch_scoring_rules_ieee14_demo_representative_v1.json \
  --week 3 --output-dir runs/demo/joint_week3
# summary: 2 teams scored, 0 rejected, scope=joint_projection
```

Together the two teams ask for 898 MW, and the grid cannot carry that, so the fair projection was used: each team is served 235.3 MW, a service fraction of **52.4%**, and scores **$2.82B** instead of $15.6B.

- Site cost is charged on the 449 MW we request, served or not.

- We are only 2.4 points above the 50% cutoff. A third team copying the same table would likely push both of us under it [...], for a score of about −$11B.
- Smaller sites, or a spread to buses 4 and 5, are much more robust to copying.

Assumptions to check yourself: [...] I mapped the columns of `hosting_capacity_mw.npy` to buses using [...] its metadata file. A wrong mapping would change the whole table and the shortlist. [...] Both runs used the default submission number. [...] The "about 44%" figure is my own back-of-envelope arithmetic, not evaluator output.

Check it yourself

Open `runs/demo/joint_week3/evaluation.json` and find the service fractions. Asking an agent to list its own assumptions is one of the most useful prompts you can write: here it flagged both the column mapping and the break-even number it had computed by hand, which turned out to be off.

Key takeaways of this session

- The agent ran every step of the workflow on its own in a few minutes, and it read the files and `--help` before acting.
- A plan that looks safe on a single-bus table can lose most of its value once someone else builds: hosting capacities do not add, and the most obvious buses attract everyone (section 2.2).
- You remain responsible for every number. Re-run one command, redo one calculation, and open one output file for every answer you use.

7 Explore the data

The representative data is in one folder, `data/processed/activsg2000_representative_v1/`. The files you are most likely to need:

File	Contents
`bus.csv`	All valid `bus_id` values.
`demand_bus.csv`	Base load at every bus in each of the 120 hourly periods.
`representative_days.csv`	The five days: date, weight (days per year), first period index, daily energy, and peak load.
`generator_catalog.csv`	Existing generators: bus, `p_min_mw`, `p_max_mw`, `marginal_cost_c1_per_mwh`, status.
`branch_limits.csv`	Lines: `from_bus_id`, `to_bus_id`, `reactance_pu`, `thermal_limit_mva`, status.
`manifest.json`	Horizon, day weights, and validation status.

This script loads the data and prints a summary:

```
from pathlib import Path
import pandas as pd

data = Path("data/processed/activsg2000_representative_v1")
buses = pd.read_csv(data / "bus.csv")
gens = pd.read_csv(data / "generator_catalog.csv")
branches = pd.read_csv(data / "branch_limits.csv")
days = pd.read_csv(data / "representative_days.csv")
demand = pd.read_csv(data / "demand_bus.csv", index_col=0)  # 120 periods x buses

print(len(buses), "buses,", len(branches), "branches,", len(gens), "generators")
```

```
print("generator capacity", round(gens.p_max_mw.sum()), "MW")
system = demand.sum(axis=1)
print("system load", round(system.min()), "to", round(system.max()), "MW; peak in period", system.
    ↪ idxmax())
print(days[["date", "weight"]].round(2).to_string(index=False))
```

Output:

```
2000 buses, 3206 branches, 544 generators
generator capacity 96292 MW
system load 58167 to 78177 MW; peak in period 109
      date  weight
2028-01-03   83.77
2028-05-10   99.73
2028-07-12   82.77
2028-09-10   97.73
2028-09-15    1.00
```

Period 109 is 1 p.m. on the peak-load day.

Without teams' plans the LMP is nearly the same everywhere, about \$18/MWh: the network is not congested. So baseline prices will not tell you where to build. What matters is where the network still has "room" once you add tens of MW. Good places to start looking are buses with strong line connections, nearby generation with spare capacity, and little base load.

Ask AI

I have these pandas DataFrames from a power-system dataset: [paste the column names of `bus_load_base.csv`, `generator_catalog.csv`, and `branch_limits.csv`]. For each bus, compute
(1) the total thermal limit of in-service branches connected to it,
(2) the total `p_max_mw` of in-service generators at that bus and at its direct neighbors, and
(3) its base load. Rank buses by a simple score of your choice, explain the score, and show the top 20.

7.1 Hosting capacity: how much room a bus has

The "room" has a name. The hosting capacity of a bus is the largest additional load that can be connected there without making the grid fail, that is, without making the DCOPF infeasible, while everything else in the network stays the same [4, 30]. Because your site draws the same load in every period, what matters is the *smallest* hosting capacity over the 120 periods, which usually occurs in the hours with the highest system load.

Here are hosting capacities for a few buses, computed with the code in appendix A for period 0 and for period 109, the hour with the highest system load:

Bus	Period 0	Period 109 (peak)
1001	391 MW	332 MW
1002	129 MW	123 MW
7419	687 MW	248 MW

For all 2,000 buses, the smallest hosting capacity over the 120 periods has a median of 371 MW; one bus in ten is below 130 MW, and 146 buses can take more than 2,000 MW. Computing all $2{,}000 \times 120 = 240{,}000$ hosting capacities takes about 20 minutes on a 32-core server with Gurobi and roughly three hours on one laptop core with HiGHS, so screen first: compute the peak hours for all buses, then all 120 hours for a shortlist.

Hosting capacity tells you where you can build and roughly how much. Use it as a screening calculation only: the official result also includes other teams.

Watch out

Hosting capacity is computed as if your site were the only new load. Other teams' sites use up the same headroom. At the peak hour, bus 1002 can host 123 MW on its own, but only about 81 MW if another team connects 60 MW at the neighboring bus 1010. The hosting capacities of different buses do not add up, and the busier a part of the network becomes, the less room each site there has (section 2.2).

Appendix A gives the optimization behind hosting capacity, how to compute it with the GitHub repository's own DCOPF solver, and how private generation and storage change it.

8 Write and check your CSV

Your submission is one UTF-8 CSV file with exactly this header and one row per site:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
```

Column	Rule
`team_id`	Your team ID. The same on every row.
`site_id`	Your own label for the site, unique within the file.
`bus_id`	Must exist in the block's `bus.csv`.
`connection_capacity_mw`	$C \geq 0$, in MW.
`generator_capacity_mw`	$0 \leq G \leq 0.5\,C$, in MW.
`storage_power_mw`	$P \geq 0$, in MW.
`storage_energy_mwh`	$B = 4P$, in MWh (both zero means no battery).

IDs may contain letters, digits, and `_ . : -`, with no spaces and at most 128 characters. The file contains capacities only: do not add operating-period rows or any other columns.

A valid two-site example:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
team-blue,site-north,1001,40,20,10,40
team-blue,site-south,1002,15,0,0,0
```

Run the checker on it. Pass `--artifact-dir` so that it also checks your bus IDs against the representative data:

```
ercot-game-check-csv --file team-blue.csv \
  --artifact-dir data/processed/activsg2000_representative_v1
```

```
VALID: team-blue.csv
team=team-blue sites=2 rows=2 buses=2 generators=1 storage=1 horizon=None
```

The exit status is 0 for a valid file, 1 for an invalid file, and 2 for a problem with the command itself. Here is an invalid file and what the checker says about it:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
team-blue,site-north,1001,-5,20,10,30
team-green,site-south,1002,15,0,0,0
team-blue,site-north,9999999,10,0,0,0
```

```
INVALID: bad.csv
- row 2: connection_capacity_mw must be non-negative
- row 2: storage energy must equal four times storage power
- row 3: one CSV may contain only one team_id
- row 4: unknown scenario bus 9999999
- row 4: duplicate site_id site-north
```

Row numbers count the header as row 1.

In Submission 2, list only new sites. Your Submission 1 sites are already part of the base load. A Submission 2 file that reuses one of your Submission 1 `site_id`s is rejected as a whole.

Ask AI

Write a Python function `write_plan(team_id, sites, path)` where `sites` is a list of dicts with keys `site_id`, `bus_id`, `connection_mw`, `generator_mw`, and `storage_mw`. It should write a CSV with exactly this header: [paste the header], set `storage_energy_mwh` to 4 times `storage_mw`, and then run `ercot-game-check-csv --file <path>` with `subprocess` and print the output.

9 Preview your plan

The preview runs an approximate version of the evaluator on your plan alone,[3] so you can compare ideas before submitting. Put your CSV in its own folder and run:

```
mkdir -p runs/candidate-a
cp team-blue.csv runs/candidate-a/
ercot-game-preview \
  --submissions-dir runs/candidate-a \
  --week 3 \
  --output runs/candidate-a.json
```

The preview uses the representative data and the ten-year rules by default. `--week3` charges energy at the LMP; use `--week1` or `--week2` to leave it out. For Submission 2, add `--artifact-dir` with the Submission 2 data your instructor releases after Submission 1: the representative data with every team's Submission 1 load already in the base load. (To try your own Submission 1 result instead, pass `--base-load-from` with a Submission 1 output folder; a Submission 2 preview without either is refused.) Every CSV in the folder is dispatched together, so keep one candidate per folder.

The preview differs from the official evaluator in two ways:

	Preview	Official evaluator
Operating periods solved	All 120 representative hours	All 120 representative hours
Storage	Not dispatched (idle), but its capital is charged	Dispatched within each representative day; half full at the start and end of each day
Other teams	Only the CSVs you put in the folder; if they exceed the grid, the same fair projection as the evaluator	All teams, with fair projection if needed

[3] It is an approximate version because running the full version is computationally extensive. If you have enough computational power, you can ask your AI agent to write a full evaluator script and simulate other teams' plans to find what is the best you can do.

So the preview usually understates the value of storage and overstates how much room the network has, because other teams are missing. Treat it as a way to rank your own candidates, not as a forecast of your score.

The output JSON has `"mode": "student_preview"` and `"exact": false`, plus one entry per team in `team_scores` with `score`, `served_load_mwh`, `it_value`, `energy_cost`, `generator_operating_cost`, `generator_capital_cost`, `storage_capital_cost`, `site_cost`, `feasible`, `service_fraction`, and `dropped`. If your plan does not fit the grid in some hour, the preview scales it back with the evaluator's fair projection, prints a NOTE, and records the details under `projection`. `served_load_mwh` is per represented year: compare it with $C \times 8{,}760$ to see how much of your load was served.

Watch out

The preview refuses to run on a scenario block whose `manifest.json` does not say `"validation_status": "passed"`. Your instructor releases blocks as they are validated.

10 Putting it together: your first plan

This section strings the previous steps together for Week 2, Submission 1. Keep each candidate in its own folder under `runs/` and log every run (appendix E).

1. Shortlist buses. Screen buses with the script in section 7, then compute hosting capacity for the shortlist over the peak-load hours (appendix A). Keep buses whose smallest hosting capacity is well above the size you have in mind.
2. Choose sizes. Start without a generator or battery. Keep each site well below its hosting capacity, and remember that other teams may be looking at the same buses.
3. Write the CSV by hand or with the `write_plan` helper from section 8, for example `runs/plan-a/plan.csv`.
4. Check it:

```
ercot-game-check-csv --file runs/plan-a/plan.csv \
  --artifact-dir data/processed/activsg2000_representative_v1
```

5. Preview it with Week 2 rules, so that there is no energy cost:

```
ercot-game-preview --submissions-dir runs/plan-a --week 2 \
  --output runs/plan-a.json
```

6. Read the result. Divide `served_load_mwh` by $C \times 8{,}760$ to get the share of your load that was served, and look at which cost term dominates `score`.
7. Change one thing (a bus, a size, or adding a generator), preview again, and compare (section 11).
8. Submit the best candidate, then read your results (section 14) before planning Submission 2.

11 Develop a strategy

There is no single right answer, but the scoring rules point to a few questions worth answering with data.

When to build? A site earns the same ten-year value in either submission, but Submission 1 sites are connected first: in Submission 2 they are part of the base load, and new sites compete only for the room that is left. Submission 2 is your chance to add sites once you have seen where everyone else built.

How big? A fully served site earns about $36.5M per MW over ten years after site cost, and breaks even at about 41% served. Grow capacity in steps and watch where served load stops keeping up.

Where? Since baseline prices are flat, location matters through network headroom. Shortlist buses with a large hosting capacity (section 7.1), keep your request comfortably below it to leave room for other teams, and preview the same capacity at several of them.

Private generation? Your generator is backup for the tight hours, not a second power supply: it may run at most two hours a day at full output (air permits limit how long on-site engines may run [26, 9]). It therefore helps only where your bus's hosting capacity dips for a few hours a day, as on the peak day; at a bus that is limited in every hour, it can cover only a small part of the shortfall. At $80/MWh or more it also costs more than baseline grid energy (about $18/MWh), so in uncongested hours it sits idle and its $2.0M/MW capital is spent for nothing. Look at how your bus's hosting capacity varies over the day before you buy one (section 7.1).

Storage? With nearly flat prices, there is little to gain from buying low and selling high. Like a generator, a battery is mainly useful at a location where the network is tight in some operating periods. Remember that the preview does not dispatch storage, so it will not show this benefit.

Other teams. Your preview runs without them, but the official dispatch includes everyone. If the combined plan is infeasible, the projection in section 2.2 cuts the largest requests first, down to a common level, and you still pay for everything you submitted.

Use Week 1. Week 1 does not count. Use it to see how your plan is treated in the joint evaluation with everyone else.

11.1 A two-team example

Here is the game in miniature. Team A builds at bus 1002 and team B at the neighboring bus 1010, and each chooses between a small (50 MW) and a large (100 MW) site (only two options for simplicity). On its own, either bus could take 100 MW: at the peak hour their hosting capacities are 123 and 135 MW. Together they cannot. Both teams build in Submission 1, and the evaluator scores them on all 120 representative hours. Each served MW is worth about $61.5M over ten years, and each built MW costs $25M.

Payoffs, A / B ($M)	B small (50 MW)	B large (100 MW)
Weeks 1–2 (no energy cost)		
A small (50 MW)	1,826 / 1,826	1,826 / 2,980
A large (100 MW)	3,172 / 1,826	1,820 / 1,820
Week 3 (energy at LMP)		
A small (50 MW)	1,772 / 1,772	1,421 / 2,207
A large (100 MW)	2,426 / 1,394	1,363 / 1,333

- Both small. The grid can serve both sites in full.
- One large, one small. The combined plan is infeasible. The small team asked for less than the common level, so it is served in full. The large team gets what is left (89.1 MW for B, 92.2 MW for A) and still pays for 100 MW. Being the only large team pays off.
- Both large. Both teams are cut to the same level, about 70.2 MW. Each pays for 100 MW but is served only 70%.
- Week 3: congestion is expensive. When the bottleneck is full, the LMP at both buses rises, and energy costs $400–770M per team instead of about $55M. Even the small team pays for the large team's crowding.

There is no plan that is best whatever the other team does.[4] If B goes small, A should go large (3,172 beats 1,826); if B goes large, A should go small (1,826 beats 1,820, and in Week 3, 1,421 beats 1,363). The same holds for B. In both weeks the game has two stable outcomes, (large, small) and (small, large): the Nash

[4] That means, for this two-player game, there is no dominant strategy for either player.

equilibria of this game. It is a game of chicken: each team wants to be the large one, but if both insist, both lose. Three lessons carry over to the real game: asking for more than your share of a shared bottleneck is not rewarded; the grid's spare capacity goes to whoever does not crowd it; and your best plan depends on what you expect the others to do.

11.2 Run controlled experiments

Change one thing at a time and record every run:

Run	Change from baseline	Served fraction	Preview score
baseline	1 site, bus 1001, 40 MW		
bigger	same bus, 60 MW		
moved	40 MW at another bus		
generator	baseline + 20 MW generator		

Ask AI

Write a Python script that takes a list of candidate plans (each a list of sites with `bus_id` and capacities), writes each plan to its own folder `runs/<name>/plan.csv` with this header: [paste header], runs this command for each folder: [paste the preview command], reads `team_scores[0]` from each output JSON, and prints a table of name, `served_load_mwh`, and `score`. Run candidates one at a time.

12 Troubleshooting

Message	What to do
`header must be exactly: ...`	Copy the header from section 8. Check for extra spaces, a byte-order mark, or a renamed column.
`submission must be UTF-8 CSV`	Save as "CSV UTF-8". Avoid saving from Excel in its native format.
`expected exactly 7 fields`	A row has a missing or extra comma.
`unknown scenario bus N`	Look up the bus in `bus.csv` of the block passed to `--artifact-dir`.
`duplicate site_id X`	Give every row its own `site_id`.
`one CSV may contain only one team_id`	Use your team ID on every row.
`... must be non-negative`	Use zero or a positive number.
`... must be a finite decimal`	Use a plain number: no units, no blanks, no text.
`generator capacity cannot exceed 0.5 x connection capacity`	Lower G to at most $0.5\,C$, or raise C.
`storage energy must equal four times storage power`	Set $B = 4P$, or both to 0.
`CONFIGURATION ERROR: ...`	The command is wrong, not your file. For example, `--artifact-dir` must point to an existing scenario folder.
`scenario block validation_status is not passed`	The preview is pointing at an unreleased block. Use a block your instructor has released.
`no valid CSV submissions were found`	The preview folder is empty or every CSV failed; the lines above it give each file's reason. Run the checker first.
`connection_capacity_mw is 0, so ...`	Set $C > 0$, or drop G and P.
`empty site: every capacity is 0`	Remove the row.
`command not found: ercot-game-...`	Activate the virtual environment and rerun the install step in section 4.
`No module named 'highspy'`	Rerun the install step; the solver is a required dependency.

If you are stuck, ask an AI tool, and give it everything it needs:

Ask AI

I am running this command: [paste]. It printed: [paste the full output]. The first three lines of my CSV are: [paste]. What is wrong and how do I fix it?

13 Before you submit

- The header is exactly the seven columns, with one row per site.
- The same `team_id` is used on every row, and every `site_id` is unique.
- Every `bus_id` exists in the block you are submitting for.
- $G \le 0.5\,C$, and storage is either $(0, 0)$ or $B = 4P$.
- For Submission 2, every Submission 1 row is unchanged.

- The checker prints `VALID` for the exact file you will upload.
- You can explain why you chose each bus and capacity.
- The file name and upload location match Rules of the game, and you upload before the deadline in Assignment at a glance.

14 After each submission: reading your results

After every submission the instructor releases:

- (private) the score breakdown: IT value, energy cost, generator operating cost, and the capital costs charged in that submission;
- (public) the service fraction α of every team;
- (public) whether the fair projection was triggered;
- (public) an interactive grid map drawn on a schematic layout of the network. Each bus is colored by the total connection capacity that all teams placed there together, and each line by how heavily it was loaded. Hover over a bus or line to see the numbers. Only totals per bus are shown, so the map does not say who built what.

Read them in this order:

1. Was the projection triggered? If not, the combined plan fit the grid and every team has $\alpha = 1$. If it was, compare your α with the other teams'. An α of exactly 0 means your request would have been served less than half and was dropped (section 2.2).
2. Is your α above break-even? A site needs about 41% served to pay for itself over ten years (section 2).
3. Which term drives your score? Usually IT value against site cost; in Week 3, check whether congestion made energy expensive.
4. How does it compare with your preview? If much less of your load was served than in the preview, the shortfall came from other teams' sites, which your preview did not include.
5. Where is the grid crowded? On the map, look for busy buses and heavily loaded lines near your sites, and for regions that still look quiet.

Planning the next submission. After Submission 1, every team's connected load becomes base load. Your instructor releases the Submission 2 data, the representative data with that load added, so you can recompute hosting capacity and preview new sites against what is actually left (appendix A). After a week, everything resets, but remember that the other teams have learned from the results too.

15 Frequently asked questions

Why is my score negative?
Your site was served less than about 41% of its request, so ten years of IT value did not cover the site cost. For example, a 50 MW site served at 30% earns $0.3 \times \$3{,}076\text{M} = \$923\text{M}$ against \$1,250M of site cost: $-\$327$M. In Week 3, congestion can also push energy costs up. Check your α and the map.

Is the preview my score?
No. It leaves batteries idle and ignores other teams (section 9). Use it to compare your own candidates. All those simplifications are for reducing computational costs. If you have enough computational power, you can write a full evaluator and test your plan.

Does Week 1 count?
No. Weeks 2 and 3 count, and both submissions in each of those weeks count (Rules of the game).

When do I pay for energy?
Only in Week 3, in both submissions, at the LMP. Weeks 1 and 2 have no energy cost.

Can I schedule my battery or sell power from my generator?
No. The evaluator dispatches both, and they can only serve your own load.

What happens to my Submission 1 sites in Submission 2?
They become part of the base load: already connected, and already scored for their ten-year value. Your Submission 2 file lists only new sites.

Can I resize or move a site in Submission 2?
No. You can only add new sites, generators, and batteries.

What happens if my file is invalid?
It scores 0 for that submission (Rules of the game). Other teams are evaluated without it. Run the checker on the exact file you upload.

Why is my α zero?
The combined plan did not fit the grid, and at the common level your team would have been served less than half of its request, so the minimum allocation dropped it (section 2.2). You still pay capital for what you built. Ask for less at congested buses, or move to buses with more room.

Can other teams see my plan?
Not in detail. They can see every team's α, and the total connection capacity at each bus, not who built what.

My generator never seems to run. Is something wrong?
Probably not. At \$80/MWh or more it costs more than grid energy at about \$18/MWh, so the operator only uses it when the grid cannot deliver enough to your bus, or when congestion pushes the price at your bus above its cost.

16 Glossary

Term	Meaning	See
ACTIVSg2000	The synthetic 2,000-bus Texas grid used in the game.	section 1
Representative data	Five typical days of 120 hourly periods that stand for the next ten years.	section 7
Bus	A node of the network; every site connects to one bus.	section 1
Site	One row of your CSV: a bus plus the capacities you build there.	section 8
Connection capacity C	The load your site draws in every hour, in MW.	section 1
Submission 1 / 2	The two rounds of a week; Submission 1 load becomes Submission 2 base load.	section 1
Capital cost	One-time cost of building a site, generator, or battery.	section 2
IT value	\$1,000 earned for each MWh of your load that is served, over ten years.	section 2
DCOPF	DC optimal power flow, the dispatch the evaluator solves for each operating period.	section 3
LMP	Locational marginal price: the cost of one more MWh at a bus.	section 3
SOC	State of charge: the energy stored in a battery.	section 3
Fair projection	How the evaluator shares the grid when not every request fits.	section 2.2
Service fraction α	The share of your requested load that is served.	section 2.2
Minimum allocation	A team that would be served less than half of its request gets nothing.	section 2.2
Hosting capacity	The most new load a bus can take without making the grid infeasible.	section 7.1
Checker	`ercot-game-check-csv`; checks your file's format.	section 8
Preview	`ercot-game-preview`; an approximate score for your plan alone.	section 9
Evaluator	The instructor's official, exact scoring run with all teams.	section 1
Nash equilibrium	A set of plans in which no team gains by changing its own.	section 11.1

Appendices

A Hosting capacity in detail

The hosting-capacity idea comes from distribution planning, where it measures how much distributed generation a feeder can absorb before its performance becomes unacceptable [4]. The same idea applies to load. Recent work uses it to ask how large a data center a transmission bus can accept [30], and treats it as something that changes over time rather than a single number [6]. In this appendix, we state a version that matches this game.

A.1 Formulation

Write the per-period DCOPF in shift-factor (PTDF) form. Let $\mathbf{d}_t$ be the vector of bus loads in operating period t, $\mathbf{s}_t$ the fixed DC-line injections, M the matrix that maps each generator to its bus, Φ the PTDF matrix (the flow on each branch per MW injected at each bus and withdrawn at the reference bus), $\mathbf{F}$ the branch thermal limits, and $\mathbf{p}^{\min}, \mathbf{p}^{\max}$ the generator limits. The hosting capacity of bus k in period t is the optimal value of the linear program [1, 30]

$$
\begin{aligned}
\mathrm{HC}_{k,t} = \max_{L,\,\mathbf{p}^{\mathrm{g}},\,\mathbf{p},\,\mathbf{f}} \quad & L \\
\text{s.t.} \quad & \mathbf{p} = M\,\mathbf{p}^{\mathrm{g}} + \mathbf{s}_t - \mathbf{d}_t - L\,\mathbf{e}_k && \text{(net bus injections, new load at } k\text{)}, \\
& \mathbf{1}^{\top}\mathbf{p} = 0 && \text{(system balance)}, \\
& \mathbf{f} = \Phi\,\mathbf{p}, \quad |\mathbf{f}| \le \mathbf{F} && \text{(DC power flow, thermal limits)}, \\
& \mathbf{p}^{\min} \le \mathbf{p}^{\mathrm{g}} \le \mathbf{p}^{\max}, \quad L \ge 0 && \text{(generator limits)}.
\end{aligned}
$$

The decision variables are the new load L, the generator outputs $\mathbf{p}^{\mathrm{g}}$, the net bus injections $\mathbf{p}$, and the branch flows $\mathbf{f}$; $\mathbf{e}_k$ is the k-th unit vector. These are exactly the DCOPF constraints of appendix B with one extra load L at bus k, and the generation-cost objective is replaced by "make L as large as possible". (The PTDF form is equivalent to the voltage-angle form $f_\ell = b_\ell(\theta_{o(\ell)} - \theta_{r(\ell)})$ that many textbooks use.) At the optimum some branch is at its thermal limit, or the generators are at their limits: that binding constraint is the bottleneck for bus k.

Over the 120 representative hours. Your site's load is the same in every period, so the number that matters for a plan is

$$\mathrm{HC}_k = \min_t \mathrm{HC}_{k,t}.$$

This equals one large LP with a single L shared by every period. In practice you first compute $\mathrm{HC}_{k,t}$ for the hours with the highest system load. The minimum over a sample of periods is an upper bound on HC_k, so check more periods before relying on a number close to the limit.

Several sites at once: the hosting-capacity region. Replace $L\,\mathbf{e}_k$ by a vector $\mathbf{x} \ge 0$ of new loads, one per bus. The set of load vectors that keep every operating period feasible is

$$
\mathcal{H} = \left\{ \mathbf{x} \ge 0 : \begin{array}{c} \forall t \;\; \exists\, \mathbf{p}_t^{\mathrm{g}}, \mathbf{p}_t : \\ \mathbf{p}_t = M\,\mathbf{p}_t^{\mathrm{g}} + \mathbf{s}_t - \mathbf{d}_t - \mathbf{x}, \;\; \mathbf{1}^{\top}\mathbf{p}_t = 0, \\ |\Phi \mathbf{p}_t| \le \mathbf{F}, \;\; \mathbf{p}^{\min} \le \mathbf{p}_t^{\mathrm{g}} \le \mathbf{p}^{\max} \end{array} \right\}.
$$

It is a convex polytope, the projection of the DCOPF feasible set onto the new loads [10]. Each single-bus hosting capacity is where $\mathcal{H}$ meets one axis, so single-bus numbers do not add up. When two buses share a bottleneck, $\mathcal{H}$ is much smaller than the box formed by their individual hosting capacities: on the game grid at the peak hour, bus 1002 hosts 123 MW alone but only 81 MW with 60 MW at its neighbor 1010. The fair

projection in section 2.2 picks a point in $\mathcal{H}$ for all teams' sites, which is why your service depends on where everyone else builds.

With a private generator. A site at bus k with load L and a private generator $0 \le g_t \le G$ shows the grid only its net withdrawal $L - g_t$. The plan is feasible if and only if, in every period, some $g_t \in [0, G]$ gives $L - g_t \le \mathrm{HC}_{k,t}$, so

$$\mathrm{HC}_k^G = \min_t \mathrm{HC}_{k,t} + G.$$

This is exact when the generator is always available; outages, ramp limits, or fuel limits make it smaller. The generator runs only when the grid is short or when its cost is below the local price, and you pay for it through capital and an operating cost of at least \$80/MWh.

With storage: the periods become coupled. A battery at bus k with power P, energy B, signed output u_t (positive means discharging), and stored energy e_t gives the multi-period LP

$$\begin{aligned} \mathrm{HC}_k^S = \max_{L,\,u,\,e} \quad & L \\ \text{s.t.} \quad & L - u_t \le \mathrm{HC}_{k,t}, \quad -P \le u_t \le P && \forall t, \\ & e_t = e_{t-1} - \Delta t\, u_t, \quad 0 \le e_t \le B && \forall t, \\ & e_0 = e_T, \end{aligned}$$

where $e_0 = e_T$ is the daily cycle (in the game, $e_0 = e_T = B/2$ with T the periods of one day). The battery discharges in tight periods and recharges in slack ones [15]. Summing the energy equation over the cycle gives $\sum_t u_t = 0$; summing the first constraint then gives

$$\mathrm{HC}_k^S \le \frac{1}{T} \sum_t \mathrm{HC}_{k,t},$$

so storage lifts the limit at most from the minimum to the average headroom of the day. Its power ($L \le \mathrm{HC}_{k,t} + P$) and energy B keep it below that bound.

Further reading: flexible connection. In this game your load is fixed: the site draws its full capacity in every operating period, so a single tight period sets HC_k. Gu, Chen, and Qin [11] ask what happens if a new load instead agrees to be curtailed in a small number of time slots per year, an arrangement called flexible connection:

$$\max_{L,\,\delta} \; L \quad \text{s.t.} \quad L - \delta_t \le \mathrm{HC}_{k,t}, \quad \delta \ge 0, \quad \|\delta\|_0 \le K,$$

where δ_t is the load curtailed in period t and $\|\delta\|_0$ counts the curtailed periods, which makes the problem nonconvex. Its solution is

$$\mathrm{HC}_k^{(K)} = \mathrm{HC}_{k,[K+1]},$$

the $(K+1)$-th smallest of the operating-period hosting capacities: you are no longer limited by the K tightest periods. Their distribution-feeder case study finds that curtailing in only 1% of the 15-minute slots of a year (about 88 hours) raises the hosting capacity by about 78%, that the gain grows more slowly as K increases (see also [20]), and that most required curtailments are shallow. Letting the load delay its consumption instead of dropping it unlocks at most as much. A follow-up paper extends the approach to grids where only some buses are measured [12].

You cannot curtail in this game, but your own generator or battery can play the same role: whatever the grid cannot deliver in the K tightest periods must come from on-site resources, that is, $C - \mathrm{HC}_{k,t}$ MW in each of those periods. To see whether that is worth it at a bus you like, compute $\mathrm{HC}_{k,t}$ for the peak periods, sort them, and compare the smallest value with, say, the 25th smallest. The difference is the extra capacity that covering the 24 tightest periods would unlock, and the gap in each of those periods tells you the generator or battery power you would need.

How much should you trust a hosting-capacity number?

- Security. The game uses a plain DCOPF with no N-1 contingency or reserve constraints; dropping N-1 overstates data-center hosting capacity by about 20% on the IEEE RTS-24 system [30].
- Dynamics. The data center's own ride-through limits (UPS, converters) can be the binding constraint [19].
- Model. DC power flow has no voltage or reactive-power limits and no losses.
- Simultaneity. Single-bus numbers ignore everyone else who is connecting at the same time; that is what the region $\mathcal{H}$ and the fair projection are for.

A.2 Computing it

You do not need to write the LP yourself. Since the set of feasible L is an interval $[0, \mathrm{HC}_{k,t}]$, a bisection that asks "is the DCOPF still feasible with L extra MW at bus k?" finds the hosting capacity. The GitHub repository's solver answers that question directly; each check takes well under a second on a laptop. (`ercot-game-hosting-capacity` solves the LP itself and is faster for many buses.) Run from the GitHub repository root:

```
from tools.activsg2000.dcopf import DCOPFInfeasible, DCOPFUnresolved, solve_prepared_period

data = "data/processed/activsg2000_representative_v1"


def feasible(bus_id, extra_mw, period):
    """True if the grid can serve extra_mw at bus_id in this period."""
    try:
        solve_prepared_period(data, period_index=period,
                              additional_demand_mw_by_bus={bus_id: extra_mw})
        return True
    except (DCOPFInfeasible, DCOPFUnresolved):
        return False  # near the limit HiGHS may not resolve; count it as "no"


def hosting_capacity(bus_id, period, upper_mw=1000.0, tol_mw=1.0):
    """Largest extra load at bus_id that keeps this period feasible."""
    lo, hi = 0.0, upper_mw
    if feasible(bus_id, hi, period):
        return hi                       # at least upper_mw
    while hi - lo > tol_mw:             # feasible loads form an interval [0, HC]
        mid = (lo + hi) / 2
        if feasible(bus_id, mid, period):
            lo = mid
        else:
            hi = mid
    return lo


peak_period = 109  # highest system load: 1 p.m. on the peak-load day
for bus in ["1001", "1002"]:
    print(bus, [round(hosting_capacity(bus, period)) for period in (0, peak_period)])
```

Output:

```
1001 [391, 332]
1002 [129, 123]
```

To find the hours with the highest system load, add up the bus loads:

```
import pandas as pd

demand = pd.read_csv(f"{data}/demand_bus.csv", index_col=0)
peak_periods = list(demand.sum(axis=1).sort_values(ascending=False).index[:24])
peak_period = peak_periods[0]
print(peak_periods[:5])
```

To check how a neighbor affects you, pass both loads at once inside `feasible`:

```
additional_demand_mw_by_bus={bus_id: extra_mw, "1010": 60}
```

The solver only runs on data that your instructor has released and validated (section 9). For Submission 2, point `data` at the Submission 2 data, which already contains every team's Submission 1 load.

Ask AI

Here is my Python function `hosting_capacity(bus_id,period)`: [paste it]. Write a script that computes it for a list of candidate buses over the 24 peak-load periods in `peak_periods`, keeps the minimum for each bus, and prints a table sorted from largest to smallest. Use `concurrent.futures` to run buses in parallel.

B The evaluator in full

This appendix writes out the problem the evaluator solves for one submission, over the 120 representative hours.

Sets and data. Buses n and branches ℓ; the matrix M that maps existing generators g to their buses, with limits $\mathbf{p}^{\min}, \mathbf{p}^{\max}$ and marginal costs $\mathbf{c}$; the PTDF matrix Φ (the flow on each branch per MW injected at each bus and withdrawn at the reference bus) and branch thermal limits $\mathbf{F}$; base loads $\mathbf{d}_t$ and fixed DC-line injections $\mathbf{s}_t$ (vectors over buses); and hourly periods $t = 1, \ldots, T$ with $T = 120$. Team i has sites s at buses $k(s)$ with capacities C_s, G_s, P_s, B_s and service fraction α_i ($\alpha_i = 1$ unless the fair projection is needed); $\mathbf{e}_n$ is the unit vector of bus n.

Decision variables, for every operating period t. Existing generation $\mathbf{p}^{\mathrm{g}}_t$; net bus injections $\mathbf{p}_t$ and branch flows $\mathbf{f}_t$; private generation in three cost segments $q^m_{s,t}$, $m = 1, 2, 3$, with costs $c^1, c^2, c^3 = 80, 150, 250$ \$/MWh and widths $w^1, w^2, w^3 = 0.25, 0.5, 0.25$; battery net output $u_{s,t}$ in MW (positive means discharging) and state of charge $e_{s,d,h}$ in MWh. Each representative day d contains 24 hourly periods $h = 1, \ldots, 24$, and period $t = 24(d-1) + h$ is the h-th of day d; $e_{s,d,h}$ is the stored energy at the end of that period and $e_{s,d,0}$ is the stored energy at the start of day d. The period length is $\Delta t = 1$ hour.

Problem. The evaluator minimizes the total generation cost over the 120 hours,

$$\min \sum_{t=1}^{T} \left(\mathbf{c}^\top \mathbf{p}^{\mathrm{g}}_t + \sum_{s} \sum_{m=1}^{3} c^m q^m_{s,t} \right)$$

(multiplying this objective by Δt would not change the dispatch), subject to, for every period $t = 24(d-1)+h$:

$$\begin{aligned}
&\mathbf{p}_t = M\,\mathbf{p}^{\mathrm{g}}_t + \sum_s \mathbf{e}_{k(s)}\Big(\sum_m q^m_{s,t} + u_{s,t} - \alpha_{i(s)}C_s\Big) + \mathbf{s}_t - \mathbf{d}_t && \text{(net injections)},\\
&\mathbf{1}^\top \mathbf{p}_t = 0 && \text{(system balance)},\\
&\mathbf{f}_t = \Phi\,\mathbf{p}_t, \quad |\mathbf{f}_t| \le \mathbf{F} && \text{(DC flow, limits)},\\
&\mathbf{p}^{\min} \le \mathbf{p}^{\mathrm{g}}_t \le \mathbf{p}^{\max}, \qquad 0 \le q^m_{s,t} \le w^m G_s && \forall s, m,\\
&e_{s,d,h} = e_{s,d,h-1} - \Delta t\, u_{s,t}, \quad 0 \le e_{s,d,h} \le B_s && \forall s,\\
&|u_{s,t}| \le P_s, \quad \Delta t\, u_{s,t} \le e_{s,d,h-1} && \forall s,\\
&\textstyle\sum_{h=1}^{24} \sum_m \Delta t\, q^m_{s,24(d-1)+h} \le 2\,G_s && \forall s, d \quad \text{(run hours)},\\
&e_{s,d,0} = 0.5B_s, \qquad e_{s,d,24} = 0.5B_s && \forall s, d,\\
&\sum_{s\in\text{team } i,\ k(s)=n} u_{s,t} \le \sum_{s\in\text{team } i,\ k(s)=n} \alpha_i C_s && \forall i, n,
\end{aligned}$$

where the last constraint keeps a team's batteries from discharging more than that team's load at a bus (a team's generators are limited the same way). Batteries and the generators' two-hour daily budget couple the 24 hours within each representative day, but not different days. The evaluator therefore solves one independent linear program per day; each day starts and ends at half charge. How it solves these daily problems quickly is the subject of appendix B.1. The grid operator (instructor) solves it exactly. The preview drops storage, which makes every operating period independent (section 9).

Prices and score. The LMP $\lambda_{n,t}$ is the cost of serving one more MW of load at bus n in period t. In the PTDF form it splits into an energy part and a congestion part,

$$\boldsymbol{\lambda}_t = \pi_t\,\mathbf{1} - \Phi^\top \boldsymbol{\mu}_t,$$

where π_t is the dual variable of the system balance and $\boldsymbol{\mu}_t = \boldsymbol{\mu}^+_t - \boldsymbol{\mu}^-_t$ collects the dual variables of the upper and lower flow limits. Without congestion ($\boldsymbol{\mu}_t = 0$) every bus has the same price π_t. When a line is at its limit, load at a bus that adds to the flow on that line ($\Phi_{\ell n} < 0$) pays more, and load at a bus that relieves it pays less. Prices are clipped at zero. Team i's grid purchase at bus n is the part of its load that its own generation and batteries do not cover, $\pi_{i,n,t} = \max\{0,\ \sum_s(\alpha_i C_s - \sum_m q^m_{s,t} - u_{s,t})\}$, summed over its sites at n. With the ten-year period weights $w_t = \Delta t \times d_t \times A$ of section 1 (d_t the days per year that period t's day stands for, $A = 7.02$),

$$\begin{aligned}
\text{IT value}_i &= 1{,}000 \sum_t w_t \sum_{s\in i} \alpha_i C_s,\\
\text{energy cost}_i &= \sum_t w_t \sum_n \lambda_{n,t}\,\pi_{i,n,t},\\
\text{gen. operating}_i &= \sum_t w_t \sum_{s\in i}\sum_m c^m q^m_{s,t},\\
\text{capital}_i &= \sum_{s\in i,\ \text{new}} \big(25\text{M}\,C_s + 2.0\text{M}\,G_s\\
&\qquad + 0.4\text{M}\,P_s + 0.25\text{M}\,B_s\big),
\end{aligned}$$

where the energy cost applies in Week 3 only, and "new" means built in this submission. Capital is not discounted. In Submission 2, the base load $\mathbf{d}_t$ also contains the hourly net grid withdrawal of every Submission 1 site, taken from the Submission 1 dispatch.

B.1 Solving a day quickly: Benders decomposition

A day on the full network is a large linear program: 24 copies of the 2,000-bus DCOPF, about 125,000 constraints, tied together only by a few devices at the teams' sites. Solving it in one piece is slow, and the

fair projection (appendix C) has to solve it many times, often for load levels that do not fit, which a solver needs a long time to prove. The evaluator therefore splits each day into a small problem about the devices and 24 ordinary hourly DCOPFs. This is Benders decomposition [2].

One important observation is: A budgeted generator can be viewed as a battery that can only discharge. A private generator with capacity G may run at most two hours a day at full output by our assumption. Seen from the grid, that is exactly a battery that starts the day holding $2G$ MWh, can discharge at most G MW in any hour, cannot recharge, and may end the day with energy left over:

$$e_0 = 2G, \qquad e_h = e_{h-1} - \Delta t\, g_h, \qquad 0 \le g_h \le G, \qquad e_{24} \ge 0.$$

The only differences are that the generator pays its fuel curve (c^m per MWh in each segment) and has no "charging" variable. So the same machinery the evaluator already uses for batteries (as discussed above) can handle the two-hour budget.

Split the day. Collect the device decisions of the day, the battery outputs $u_{s,h}$ and the generator segments $q^m_{s,h}$, in a vector x, and write X for the set allowed by the daily constraints above (state of charge, energy budgets, power limits, and "serve only your own load"). Once x is fixed, each hour is a DCOPF in which the devices are fixed injections at their buses:

$$\phi_h(x) = \min_{\mathbf{p}^{\mathrm{g}}_h} \ \mathbf{c}^\top \mathbf{p}^{\mathrm{g}}_h \quad \text{s.t. the network constraints of hour } h,$$

the cost of running the grid in hour h (with a large penalty on any load that cannot be served, so ϕ_h is always defined). The day's problem is then

$$\min_{x \in X} \ \sum_{s,h,m} c^m q^m_{s,h} + \sum_{h=1}^{24} \phi_h(x).$$

Each ϕ_h is convex and piecewise linear in x, but we do not have a formula for it, only the ability to evaluate it by solving one hourly DCOPF.

Learn ϕ_h from cuts. Solving the DCOPF of hour h at a trial point $\hat{x}$ gives its cost $\phi_h(\hat{x})$ and the LMPs at the site buses. The LMPs say how the cost changes if a device injects one more MW, so they give a linear lower bound, a cut, that holds for every x:

$$\phi_h(x) \ \ge \ \phi_h(\hat{x}) - \boldsymbol{\lambda}_h(\hat{x})^\top (x_h - \hat{x}_h),$$

where x_h are the device injections in hour h. The small master problem replaces each ϕ_h by a variable θ_h that must lie above all cuts found so far:

$$\min_{x \in X,\ \theta} \ \sum_{s,h,m} c^m q^m_{s,h} + \sum_h \theta_h \quad \text{s.t.} \quad \theta_h \ge \text{every cut of hour } h.$$

The algorithm alternates:

1. Solve the master problem. Its value is a lower bound on the day's cost, and its solution is the next trial point $\hat{x}$.
2. Solve the 24 hourly DCOPFs at $\hat{x}$ (independently, in parallel). Their total cost plus the generators' fuel is an upper bound: it is the cost of an actual dispatch.
3. Add one new cut per hour to the master and repeat until the two bounds agree (to a relative gap of 10^{-7}).

Because every ϕ_h is piecewise linear, the loop ends after finitely many cuts at the exact optimum of the daily LP; on the game's package it typically needs about 20 iterations. The master is small (it has only the devices' variables), and each hourly DCOPF is the same size as a hosting-capacity LP, so the whole day solves in seconds instead of the many minutes the one-piece LP can take. The LMPs of the full day are recovered from the master's cut multipliers.

Why this matters for you

You do not need Benders decomposition to plan: hosting capacity, generators and batteries can be reasoned about bus by bus as in appendix A. But the idea is worth knowing. "Fix the few coupling decisions, solve many small independent problems, and feed their prices back as cuts" is how large planning and scheduling problems are solved in practice.

C The fair projection in detail

When the combined plan of all valid teams is infeasible, the evaluator reduces service. Let $C_{i,s}$ be the connection capacity of team i at site s and $r_i = \sum_s C_{i,s}$ team i's total request (MW). The evaluator chooses one service fraction $\alpha_i \in [0, 1]$ per team.

C.1 How a team's service is split among its sites

Every site of a team is scaled by the same fraction

Site s of team i is served

$$\alpha_i \, C_{i,s} \text{ MW in every operating period,}$$

so the team is served $\sum_s \alpha_i C_{i,s} = \alpha_i r_i$ MW in total. The evaluator does *not* decide how to divide a team's allocation among its sites: the split is fixed by the team's own requests. Private generators and batteries are not scaled.

For example, a team that requests 60 MW at bus A and 40 MW at bus B ($r_i = 100$ MW) and receives $\alpha_i = 0.7$ is served 42 MW at A and 28 MW at B. Even if bus B is the congested one, the evaluator does not move the team's 70 MW toward A. Three consequences:

- Your mix of sites is your decision. Moving MW between your own sites happens in your CSV, not in the evaluator.
- One congested site cuts all of your sites. The rule below compares teams by their total MW. If one of your sites sits behind a bottleneck, every one of your sites is cut by the same fraction, including those at quiet buses. Keep risky and safe sites in mind when you plan.
- An oversized site hurts others. Inflating one site at a congested bus shifts the team's fixed mix toward that bottleneck, which can lower the common level for every team. The minimum allocation (step 3) removes such requests.

C.2 The four steps

1. Common level. Find the largest level t such that serving every team $\min(r_i, t)$ is feasible in every operating period. Teams with $r_i \le t$ are served in full.
2. Water-filling. A team that cannot grow on its own, even by a tiny amount, is *blocked* and keeps its level. The teams that are neither blocked nor fully served raise their common level again, and the step repeats until every team is fully served or blocked. Each level is found by bisection to within 0.1 MW.
3. Minimum allocation. If some team is served less than half of its request, $\alpha_i < 0.5$, the team with the largest request among them (ties broken by team ID) is dropped to $\alpha_i = 0$, and steps 1–2 are repeated for the remaining teams. This continues until every remaining team gets at least half of its request.
4. Fallback. If no joint solution is found, the evaluator tries leaving one team out at a time, then each team alone, and records what happened.

C.3 Steps 1 and 2 in detail

The feasibility test. Both steps ask one question many times: for given fractions $\alpha = (\alpha_1, \ldots, \alpha_n)$, is the plan feasible? To answer it, the evaluator scales every site to $\alpha_i C_s$ and runs the full dispatch of appendix B over all 120 hours, storage included. The answer is "yes" if a dispatch exists in every operating period. The search below assumes that lowering any team's fraction never makes a feasible plan infeasible.

Levels in MW. Steps 1 and 2 work with a level t in MW. At level t, team i is served $\min(r_i, t)$, that is,

$$\alpha_i(t) = \min\Big(1,\ \frac{t}{r_i}\Big).$$

Teams that are already frozen (blocked or fully served) keep their fraction while the others move.

Step 1: the common level. All teams that are not frozen are raised together. If serving all of them in full is feasible, they are done. Otherwise the evaluator searches for the largest feasible level by bisection: it keeps a level t_{lo} known to be feasible and a level t_{hi} known to be infeasible, tests the midpoint, and replaces one of them, until $t_{\text{hi}} - t_{\text{lo}}$ is at most the tolerance $\tau = \min(0.1\,\text{MW},\ 0.1\%$ of the largest request). The level becomes t_{lo}, and every team with $r_i \le t_{\text{lo}}$ is now fully served.

Step 2: water-filling. At the common level, some teams may still be able to grow: their sites are not behind the line that stopped the level. The evaluator tests each remaining team alone: it raises only that team to $t_{\text{lo}} + 2\tau$ and runs the feasibility test.

- If the test fails, the team is blocked: it is frozen at its current fraction.
- If the test passes because the team has reached its full request, the team is served in full and frozen.
- Otherwise the team can still grow, and it stays active.

The active teams then return to step 1, starting from the current level, and rise together again. The loop ends when no team is active.

The algorithm. In Python-like pseudocode (the real code is `_fair_continuous_projection` in `src/ercot_game/batch_evaluation.py`):

```
def water_fill(requests, feasible, tol):
    # requests[i] = r_i in MW; feasible(alpha) runs the full dispatch
    alpha = {i: 0.0 for i in requests}
    active = {i for i in requests if requests[i] > 0}
    level = 0.0

    def at(t, teams):                       # raise these teams to level t
        a = dict(alpha)
        for i in teams:
            a[i] = max(alpha[i], min(1.0, t / requests[i]))
        return a

    while active:
        top = max(requests[i] for i in active)
        if feasible(at(top, active)):       # Step 1: everyone active fits
            level = top
        else:                               # Step 1: bisection on the level
            lo, hi = level, top
            while hi - lo > tol:
                mid = (lo + hi) / 2
```

```
            if feasible(at(mid, active)):
                lo = mid
            else:
                hi = mid
        level = lo
    alpha = at(level, active)
    active = {i for i in active if alpha[i] < 1.0}    # fully served leave

    blocked, filled = set(), set()     # Step 2: test each team alone
    for i in sorted(active):
        trial = at(level + 2 * tol, {i})
        if not feasible(trial):
            blocked.add(i)
        elif trial[i] >= 1.0:          # reached its full request
            full = dict(alpha)
            full[i] = 1.0
            if feasible(full):
                alpha = full
                filled.add(i)
    if not blocked and not filled:
        break                          # cannot happen on a convex region
    active -= blocked | filled
return alpha
```

Step 3 calls `water_fill` again without the dropped team, until every remaining team gets at least half of its request.

A worked example. Three teams with one site each: A requests 60 MW, B and C request 40 MW each. Suppose the network allows at most 50 MW at B and C together (they share a line) and at most 45 MW at A.

Step	What happens	A	B	C
1	All rise together; at 25 MW the B–C line is full	25	25	25
2	Test alone: B and C are blocked, A can still grow	active	blocked	blocked
1	A rises alone until its own limit, 45 MW	45	25	25
2	Test alone: A is blocked; no team is active	blocked		
3	Shares 45/60, 25/40, 25/40 are all at least 50%			
	Served (MW) and α_i	45 (0.75)	25 (0.625)	25 (0.625)

If C had asked for 60 MW instead, step 1 would still stop at 25 MW, but C would then get only $25/60 = 42\%$ of its request. Step 3 drops C, and the rerun serves A 45 MW and B its full 40 MW: C's oversized request ends up costing only C.

C.4 Why the rule is max-min fair and strategy-proof

Steps 1–2 are the uniform rule [22] applied to the network. This subsection explains its two defining properties and where they stop holding. Write $s_i = \alpha_i r_i$ and $\mathcal{S}$ for the set of feasible service vectors s (those with $0 \le s_i \le r_i$ that the network can serve).

Max-min fairness. A feasible s^* is max-min fair if no team can be given more without taking something from a team that has the same amount or less: for every feasible s with $s_i > s_i^*$, some team j has $s_j^* \le s_i^*$ and $s_j < s_j^*$. Equivalently, sort the allocations from smallest to largest: no feasible allocation has a larger smallest value, or, when those tie, a larger second-smallest, and so on. The worst-off team comes first, then the next.

Water-filling builds exactly this allocation. In the first round all teams rise together until the common level t_1 cannot grow. Each team that is blocked at t_1 sits on a binding constraint, and the only other teams on that constraint are at the same level t_1 or already fully served below it. Giving a blocked team more therefore means taking from a team with no more than it has, so no feasible allocation lifts the smallest level above t_1 without hurting someone at or below it. The blocked teams are frozen, and the same argument applied to the remaining teams shows that the next level t_2 is the best possible second tier, and so on. On a compact, convex feasible set a max-min fair allocation exists and is unique, and water-filling computes it whenever lowering one team's share never makes a feasible allocation infeasible [18]. The same property guarantees that when the common level stops, at least one active team is blocked on its own, which is why testing each team alone in step 2 is enough.

Strategy-proofness. Fix the other teams' requests, and let t_i^* be the MW team i would be served if it asked for an unlimited amount. On a single shared limit H, team i is served

$$s_i = \min(r_i,\ t_i^*)\ :$$

if $r_i \le t_i^*$ it is served in full, and if $r_i > t_i^*$ the common level solves $\sum_{j \ne i} \min(r_j, t) + t = H$, which does not involve r_i, so it is served t_i^* whatever it asked. Asking for more than you want never increases your service, and asking for less only reduces it. If a team wants q MW, asking for q gets it $\min(q, t_i^*)$, the closest amount to q it can obtain, so telling the truth is a dominant strategy. Sprumont showed that the uniform rule is the only rule that is strategy-proof, efficient, and treats teams symmetrically [22]. On the network with one site per team the same structure holds: the water-filling follows the same path until team i stops, so team i is served $\min(r_i, \tau_i)$, where τ_i is the point at which it becomes blocked given the others.

In this game you pay capital on what you *request*, so the request you want is the MW you will be served. Since service is $\min(r_i, t_i^*)$, every MW requested above t_i^* only adds site cost, and a request above $2t_i^*$ is dropped by the minimum allocation. The rule follows ERCOT's batch study for large loads in the same spirit: capacity is allocated across a batch of requests studied together rather than first come, first served, and a request that cannot receive a minimum share of what it asked for receives nothing [8].

Where the guarantees stop.

- Energy prices. Strategy-proofness concerns *service*. Your score also includes energy at the LMP, which depends on your own load. At a congested bus, asking slightly less can lower your price by more than the service you give up, so truth-telling is no longer dominant.
- Several sites per team. The fraction applies to your whole mix of sites. Changing the mix, for example inflating one site at a congested bus, changes the shape of $\mathcal{S}$ and can lower the common level for everyone. Strategy-proofness covers the size of your total request, not the choice of sites; the minimum allocation is what removes such requests.
- Equal MW, not efficiency. The rule counts a MW at a bus that barely uses a bottleneck the same as a MW at a bus that uses it heavily. This is why, in the two-team example of section 11.1, both large teams are cut to the same level, and why the total served can fall short of what a planner would achieve. Kelly's proportional fairness [14], which maximizes $\sum_i \log(\alpha_i r_i)$, is reported in the results as a diagnostic.
- Discrete sizes. With only a few allowed sizes, the best request may not be available, and best responses can jump between sizes; this produces the chicken-style equilibria of the two-team example.

D Where the numbers come from

The numbers are instructional, informed by public sources and rounded. They are not tariffs, forecasts, or ERCOT rules.

Quantity	Value	Basis
IT value	\$1,000/MWh	An instructor-chosen net margin per MWh for an AI data center, between colocation rent (about \$2.4M per MW per year, or \$270/MWh [3]) and projected GPU-cloud revenue (about \$15M per MW per year, or \$1,700/MWh [7]).
Planning horizon	10 years at 7%	A data center runs for about ten years; per-MWh terms are discounted with the annuity factor $A = \sum_{y=1}^{10} 1.07^{-y} = 7.02$. The 7% rate is the one OMB Circular A-4 (2003) recommended to approximate the average return on private capital [27]; the 2023 revision lowered the default to 2% [28], but private investors in data centers expect higher returns, so 7% is a reasonable instructional choice.
Representative days	5 days, 120 hours	Four medoid days from clustering the daily load profiles of one year of hourly ACTIVSg2000-based data, plus the peak-load day; weights sum to 365.
Site cost	\$25M/MW	Cost of building an AI data center per MW of IT load: about \$10.7M/MW for the building and up to \$25M/MW with AI fit-out [13], or \$21–31M/MW including servers [16].
Generator capital	\$2.0M/MW	Installed cost of on-site gas engines and aeroderivative turbines for data centers, about \$1,700–2,000/kW [17]; EIA reference costs are in the same range [24].
Generator cap	$G \leq 0.5\,C$	Texas SB 6 (2025) treats on-site generation that can serve at least half of a large load as backup that ERCOT may call on in emergencies [29].
Generator run hours	2 h per day	Air permits limit on-site engines: federal rules allow 100 hours a year of maintenance and testing, of which at most 50 may be non-emergency use [26, 21], and state permits cap total operation, e.g. below 500 hours a year [9]. Two hours a day (about 730 hours a year) is a teaching choice in that range.
Generator operating cost	\$80/150/250 per MWh	A rising cost curve for on-site gas generation with a diesel-backup premium; the exact shape is instructional [25].
Storage capital	\$400k/MW + \$250k/MWh	Rounded from the 2028 moderate projection for four-hour batteries [5]; \$1.4M per MW of four-hour storage.
Grid energy price	LMP from the dispatch	The bus-level dual price of the DCOPF; about \$17–20/MWh before students add load.
Network	ACTIVSg2000	Public synthetic Texas grid [23]; the load profiles are synthetic, not historical data.

E Working as a team

How to collaborate: use a private Git repository. Commit your CSVs, scripts, and run log so everyone works from the same files and you can always return to the exact plan you submitted. Keep the repository private: your plan is your competitive information.

```
# illustrative: create a private repository on GitHub first, then
git clone git@github.com:<you>/team-blue.git && cd team-blue
git add runs/ scripts/ runlog.csv
git commit -m "Week 2 Submission 1: plan-a, 2 sites"
git push      # to a private repository shared only with your team
```

Keep one candidate per folder. The preview dispatches every CSV in a folder together, so give each candidate its own folder:

```
runs/
  week2-sub1/plan-a/plan.csv
  week2-sub1/plan-a.json        <- preview output
  week2-sub1/plan-b/plan.csv
runlog.csv
```

Large preview outputs need not be committed; the CSV and the command are enough to recreate them.

Log every run. One row per run is enough:

Date	Block	Candidate	Change	Served	Score	Note
10-20	2028 jan_jun	plan-a	baseline	97%	...	
10-20	2028 jan_jun	plan-b	site 2: 40→60 MW	81%	...	below break-even

Split the work. For example, one person screens buses and computes hosting capacity, one writes and checks CSVs, and one runs previews and keeps the log. Review the final CSV together before you submit.

References

[1] Mansoor Alturki, Amin Khodaei, Aleksi Paaso, and Shay Bahramirad. Optimization-based distribution grid hosting capacity calculations. *Applied Energy*, 219:350–360, 2018.

[2] J. F. Benders. Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4(1):238–252, 1962.

[3] Bitdeer Technologies Group. Form 6-k, exhibit 99.1. U.S. Securities and Exchange Commission, 2026.

[4] Math H. J. Bollen and Fainan Hassan. *Integration of Distributed Generation in the Power System*. Wiley-IEEE Press, Hoboken, NJ, 2011.

[5] Wesley Cole and Abhishek Karmakar. Cost projections for utility-scale battery storage: 2023 update. Technical report, National Renewable Energy Laboratory, 2023. The 2028 moderate four-hour projection is 351 dollars per kWh in 2022 dollars.

[6] Tiago E. C. de Oliveira, Math H. J. Bollen, Paulo F. Ribeiro, Pedro M. S. de Carvalho, Antônio C. Zambroni, and Benedito D. Bonatto. The concept of dynamic hosting capacity for distributed energy resources: Analytics and practical considerations. *Energies*, 12(13):2576, 2019.

[7] Digi Power X Inc. Form 8-k, exhibit 99.1. U.S. Securities and Exchange Commission, 2026.

[8] ERCOT. Trending topics: ERCOT's new batch connection process for large electricity users, June 2026.

[9] Georgia Environmental Protection Division. Draft air quality permit for a data center (emergency generators), 2025.

[10] Sicheng Gong and J. F. G. Cobben. Adaptive facet selection in multidimensional hosting capacity region assessment. *Electric Power Systems Research*, 234:110654, 2024.

[11] Nan Gu, Ge Chen, and Junjie Qin. The role of flexible connection in accelerating load interconnection in distribution networks. In *Proceedings of the Hawaii International Conference on System Sciences (HICSS)*, 2026. arXiv:2510.11476.

[12] Nan Gu, Vassilis Kekatos, and Junjie Qin. Flexible connection to accelerate load interconnection in partially measured distribution grids. In *Proceedings of the 17th ACM International Conference on Future and Sustainable Energy Systems (e-Energy)*, 2026.

[13] Jones Lang LaSalle IP, Inc. 2026 global data center outlook, 2026. Data-center shell/core and AI fit-out construction-cost context.

[14] Frank P. Kelly, Aman K. Maulloo, and David K. H. Tan. Rate control for communication networks: Shadow prices, proportional fairness and stability. *Journal of the Operational Research Society*, 49(3):237–252, 1998.

[15] L. H. Macedo, J. F. Franco, R. Romero, M. A. Ortega-Vazquez, and M. J. Rider. Increasing the hosting capacity for renewable energy in distribution networks. In *IEEE PES Innovative Smart Grid Technologies Conference (ISGT)*, 2017.

[16] Morgan Stanley Research. Bridging the data center gap, 2025. AI data-center construction-cost range including GPUs and servers.

[17] Rabobank. The sprint: Data centers are building a parallel energy system in the US, 2026.

[18] Božidar Radunović and Jean-Yves Le Boudec. A unified framework for max-min and min-max fairness with applications. *IEEE/ACM Transactions on Networking*, 2007.

[19] Pengyu Ren, Wei Sun, and Fei Teng. Hosting capacity assessment of data centers with voltage ride-through capability in power systems. arXiv:2609.03030, 2026.

[20] Gobinda Chandra Sarker and Nathan Dahlin. Risk-aware hosting capacity analysis for flexible load interconnection in distribution networks. arXiv:2604.20185, 2026.

[21] Sidley Austin LLP. US EPA issues new guidance on data center emergency generator operations, 2025.

[22] Yves Sprumont. The division problem with single-peaked preferences: A characterization of the uniform allocation rule. *Econometrica*, 59(2):509–519, 1991.

[23] Texas A&M University. Activsg2000 synthetic test case, 2019. Public synthetic transmission-network model; consult the repository's released scenario package for this tutorial.

[24] U.S. Energy Information Administration. Capital cost and performance characteristics for utility-scale electric power generating technologies, 2024. Technology-cost reference for the generator capital anchor.

[25] U.S. Energy Information Administration. How much electricity is generated by natural gas-fired power plants?, 2025. Heat-rate context for the instructional team-generator curve.

[26] U.S. Environmental Protection Agency. 40 CFR 63.6640(f): Requirements for emergency stationary RICE. Electronic Code of Federal Regulations.

[27] U.S. Office of Management and Budget. Circular A-4: Regulatory analysis, 2003. Recommends discount rates of 3% and 7%; 7% approximates the average before-tax return on private capital.

[28] U.S. Office of Management and Budget. Circular A-4: Regulatory analysis (revised). 88 Fed. Reg. 77615, 2023. Lowers the default rate to 2% and drops the 7% rate.

[29] Utility Dive. Texas law gives grid operator power to disconnect data centers during crisis, 2025.

[30] Haoxiang Wan, Linhan Fang, Jonghwan Kwon, Jesus Silva-Rodriguez, Jianyi Yang, Jaya Yellajosula, Fan Jiang, and Xingpeng Li. Impact of security modeling fidelity on hyperscale data center hosting capacity assessment. arXiv:2610.18405, 2026.

[31] Ray D. Zimmerman and Daniel Murillo-Sánchez. *MATPOWER User's Manual*, 8.1 edition, 2024. Case-format and power-flow conventions; accessed 2026-10-11.